Software Requirements Specification for Lake: Distributed Large Language Model Inference on Pooled Local Devices

Team #10, CinqGarçons
Ethan Walsh
Colin Chambachan
Matthew Nesbitt
Jinwoo Hong
Gary Qin

October 20, 2026

Table 1: Revision History

Date	**Developer(s)**	**Change**
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

Contents

Changes from Original SRS Meyer Template **1**

Glossary **3**

(G) Goals **9**
G.1 Context and Overall Objective . . . 9
G.2 Current Situation . . . 10
G.3 Expected Benefits . . . 11
G.4 Functionality Overview . . . 14
G.4.1 Functional Requirements . . . 14
G.4.2 Non-Functional Requirements . . . 16
G.4.3 RACI Matrix . . . 17
G.5 High-Level Usage Scenarios . . . 18
G.6 Limitations and Exclusions . . . 21
G.7 Stakeholders and Requirements Sources . . . 22
G.7.1 Stakeholder Definition . . . 22
G.7.2 Requirements Sources . . . 25
G.7.2.1 Project documents. . . . 25
G.7.2.2 Peer review. . . . 26
G.7.2.3 Teaching assistant. . . . 26
G.7.2.4 User interview. . . . 26
G.7.2.5 Regulation and guidance. . . . 26
G.7.2.6 The team as users. . . . 27
G.7.2.7 Existing distributed inference tools. . . . 27
G.7.2.8 Libraries and formats. . . . 27
G.7.2.9 Course material and the template. . . . 27
G.8 User Profiles . . . 27
G.8.0.1 Local AI user . . . 27
G.8.0.2 Operator in a regulated organisation . . . 28
G.8.0.3 Researcher . . . 28

(E) Environment **29**
E.1 Glossary . . . 29
E.2 Environment Components . . . 29
E.3 Constraints . . . 29
E.4 Assumptions . . . 30
E.5 Effects . . . 31

E.6 Invariants . . . 31
E.7 Standards and Legal/Regulatory Factors . . . 32
Environmental Requirements . . . 33

(S) System **34**
S.1 Components . . . 34
S.2 Functionality . . . 35
S.2.1 Coordinator . . . 35
S.2.2 Model Partitioner . . . 38
S.2.3 Inference Engine . . . 38
S.2.4 Tensor I/O . . . 39
S.2.5 Web UI . . . 39
S.2.6 Non-Functional Requirements . . . 40
S.2.6.1 Usability . . . 40
S.2.6.2 Performance . . . 40
S.2.6.3 Look and Feel . . . 41
S.2.6.4 Fault-Tolerance . . . 41
S.2.6.5 Privacy . . . 41
S.2.6.6 Compatibility . . . 42
S.3 Interfaces . . . 42
S.4 Normal Operation . . . 42
S.5 Undesired Event Handling . . . 43
S.6 Detailed Usage Scenarios . . . 44
S.7 Prioritization . . . 55
S.8 Verification and Acceptance Criteria . . . 63
S.8.1 Validation . . . 63
S.8.2 Verification . . . 63
S.9 Formalisations . . . 67
S.9.1 Notation . . . 67
S.9.2 Valid Partitioning Schemes . . . 68
S.9.3 Allocation State Machine . . . 68
S.9.3.1 States. . . . 69
S.9.3.2 Events. . . . 69
S.9.3.3 Guards. . . . 69
S.9.3.4 Transitions. . . . 70
S.9.3.5 Invariant. . . . 71
S.10 Likely Changes to Requirements . . . 71
S.11 Unlikely Changes to Requirements . . . 71

(P) Project **73**
P.1 Roles and Personnel . . . 73
P.2 Imposed Technical Choices . . . 73
P.3 Schedule and Milestones . . . 74
P.4 Tasks and Deliverables . . . 74
P.5 Required Technology Elements . . . 76
P.6 Risk and Mitigation Analysis . . . 77
P.7 Requirements Process and Report . . . 78

Safety and Security Requirements **80**

Requirement Dependencies **82**

References **83**

1 Appendix — User Interview **83**

2 Appendix — Symbolic Constants **84**

3 Appendix — Generative AI Use Disclosure **87**
3.1 AI Tools Used . . . 87
3.2 How and Where Used . . . 87
3.3 Verification of AI-Generated Content . . . 88

4 Appendix — Reflection **89**

List of Tables

1 Revision History . . . i
2 Changes from Template . . . 2
3 Lake compared with the current options . . . 11
4 RACI matrix for the high-level requirements . . . 17
5 RACI legend . . . 18
6 Stakeholders . . . 22
7 Environmental Requirements . . . 33
8 Prioritisation of Coordinator requirements on all nodes . . . 55
9 Prioritisation of Coordinator requirements on the leader node . . . 56
10 Prioritisation of Coordinator requirements on follower nodes . . . 57
11 Prioritisation of Model Partitioner requirements . . . 57

12 Prioritisation of Inference Engine requirements . . . 58
13 Prioritisation of Tensor I/O requirements . . . 58
14 Prioritisation of Web UI requirements . . . 59
15 Prioritisation of non-functional requirements . . . 60
16 Prioritisation of environmental requirements . . . 61
17 Prioritisation of safety and security requirements . . . 62
18 Symbols used in the formalisation . . . 67
19 Transition function δ of the allocation state machine . . . 70
20 Project Milestones and Due Dates . . . 74
21 Tasks and Deliverables . . . 75
22 Constants used in the system requirements . . . 85
23 Constants used in the safety requirements and verification . . . 86

List of Figures

1 Context diagram . . . 10
2 Goal model . . . 12
3 Use case diagram . . . 19
4 Component diagram. . . . 34

Changes from Original SRS Meyer Template

This document follows the requirements template in Meyer's *Handbook of Requirements and Business Analysis* (**?**), which organises a specification into four chapters: Goals (G), Environment (E), System (S), and Project (P). Meyer's template is a book chapter rather than a course template, so we have adapted it. Table 2 records each structural change and the reason for it. The LaTeX structure is adapted from a Meyer-template specification written by a previous capstone team and shared with us by our teaching assistant.

Table 2: Changes from Template

Change	**Reason**
Glossary (E.1) moved to the front of the document	Terms and acronyms must be defined before first use, and information should live in one place. E.1 now points here.
Requirement tables added to each PEGS chapter	Meyer's template describes requirements in prose. Labelled tables of functional (FR) and non-functional (NFR) requirements give each requirement an ID that can be prioritised, traced and verified.
Added G.8 User Profiles	Meyer lists stakeholders (G.7) but has no place for the distinct characteristics of each user group.
Added E.7 Standards and Legal/Regulatory Factors	Meyer's Environment chapter has no explicit home for standards, licensing or legal constraints.
Added S.4 Normal Operation and S.5 Undesired Event Handling	Separates expected behaviour from error handling instead of mixing both into S.2 Functionality.
Added S.9 Formalisations	Provides a place for mathematical or formal specification of the parts of the system that benefit from it.
Added Safety and Security Requirements	Lets the requirements from the Hazard Analysis be ported into this document.
Added Requirement Dependencies	Shows traceability between requirements visually.
Added References	Meyer's template has no references section.
Added Appendices (Generative AI Use Disclosure, Reflection)	Required by the course.
Added S.10 Likely changes to requirements	Required by the SRS rubric.
Added S.11 Unlikely changes to requirements	Required by the SRS rubric.
Added Appendix (User Interview)	Records the questions put to a potential user and their answers, so the requirements sources in G.7 can be checked.
Added a Symbolic Constants appendix	The SRS checklist asks for literal values to be given as named constants, listed in a table in an appendix.

Glossary

Terms and acronyms used in this document, in alphabetical order. Terms in the text link back here.

- **Activation** – An intermediate value that a model layer computes and passes to the next layer (FR.G.4.3).
- **AI (artificial intelligence)** – Software that does tasks that normally need human judgement. In this document it always means an LLM (Section G.7.1).
- **API (application programming interface)** – A set of requests one program can make of another. Lake's API is the OpenAI-compatible API on the leader node (Section S.3).
- **ARM** – One of the two CPU architectures Lake supports, used by Apple silicon and most mobile devices (NFR.S.14).
- **Backpressure** – A signal that a node is receiving tensors faster than it can process them, so the upstream node should slow down (FR.S.1.2).
- **Bottleneck** – The node or network connection that takes the longest per token and limits the throughput of the whole cluster (UC7, FR.S.5.6).
- **Cluster** – The nodes on one local network that have joined to serve one model. Also called the device pool or the Lake network (FR.G.4.1).
- **Context window** – The maximum number of tokens, prompt and response together, that a model can handle in one request (NFR.S.4).
- **Continuous integration** – Building and testing the code automatically on every push to the repository (Section P.5).
- **Coordinator** – The Lake component that forms and monitors the cluster, serves the API on the leader node, and manages the Inference Engine on each node (Section S.1).
- **CPU (central processing unit)** – A device's general-purpose processor (FR.S.4.2).
- **Device owner** – The person whose computer joins a cluster. They must consent before their device takes part (Section E.2).

- **Device pool** – The collection of nodes that make up a cluster (Section E.2).
- **Dynamic allocation** – Moving model layers between nodes while the cluster is running, when a node joins, leaves, or fails (FR.G.4.4, Section S.9.3).
- **FFI (foreign function interface)** – An interface that lets code in one language call code in another (Section S.3).
- **FLOP/s (floating-point operations per second)** – The number of floating-point operations a processor performs per second, used to measure compute performance (FR.S.2.4).
- **Follower node** – Any node in a cluster other than the leader node (Section S.1).
- **GGUF** – The file format used by llama.cpp and ggml to store a model's weights, tokeniser, and metadata. Lake only accepts GGUF models (FR.G.4.2).
- **GPL (GNU General Public License)** – The free software licence, version 3.0, that Lake is released under (Section E.7).
- **GPU (graphics processing unit)** – A device's parallel processor, which does the arithmetic of inference (NFR.S.15).
- **Greedy decoding** – Always choosing the most probable next token, so the same prompt gives the same output every time (Section S.8).
- **gRPC** – A remote procedure call framework from Google. Tensor I/O uses it to send tensors between nodes (Section S.3).
- **HTTP (Hypertext Transfer Protocol)** – The request and response protocol of the web (FR.S.1.9).
- **Inference** – Using a model to generate text from a prompt.
- **Inference Engine** – The Lake component on each node that loads the node's partition and runs it to generate output. It is built on ggml (Section S.1).
- **Inference user** – A person, or a tool acting for one, who sends prompts to a cluster and gets responses back (Section E.2).
- **IPC (inter-process communication)** – Passing data between processes on the same machine (Section P.4).

- **KV cache** – The key and value tensors each layer stores for past tokens so they are not recomputed. Each node keeps the KV cache for its own layers (FR.G.4.3).
- **Lake** – The system this document specifies. It runs one LLM across several devices on a local network using model layer partitioning (Section G.1).
- **Lake client** – Lake running on one device. Every node runs one. The leader node's client also serves the web UI and the API (Section S.4).
- **Latency** – The time between the start of an inference request and a defined event, such as the first token or the end of the response.
- **Layer** – One of the stacked stages of a transformer model. Lake splits a model between nodes at layer boundaries (Section G.1).
- **Leader node** – The node whose operator formed the cluster. It runs the Model Partitioner, serves the API and the web UI, and monitors the followers (Section S.1).
- **LLM (large language model)** – An AI model trained on a large body of text and used to generate text.
- **Model layer partitioning** – Splitting an LLM's layers into consecutive ranges and giving each range to a different node (FR.G.4.2).
- **Model Partitioner** – The Lake component on the leader node that decides which layers each node holds, based on each node's memory, compute, and network performance (Section S.1).
- **MoSCoW** – A prioritisation scheme that sorts requirements into Must have, Should have, Could have, and Won't have (Section S.7).
- **NIST (National Institute of Standards and Technology)** – The United States standards agency (Section G.7.2).
- **Node** – A device running the Lake client that has joined a cluster.
- **OCR (optical character recognition)** – Converting images of text into machine-readable text (Section G.3).

- **OpenAI-compatible API** – An HTTP API with the same request and response formats as the OpenAI API, so clients written for OpenAI work with Lake unchanged (FR.G.4.6, Section S.3).
- **Operator** – The person who forms a cluster, loads a model, and monitors performance from the web UI. Also called the Lake operator (Section E.2).
- **P2P (peer-to-peer)** – A direct connection between two nodes that does not go through the leader. Neighbouring nodes send tensors this way (Section S.4).
- **Partition** – A consecutive range of a model's layers assigned to one node (Section S.1).
- **Partitioning scheme** – The assignment of partitions to nodes for a whole model (Section S.9.2).
- **PEGS** – Meyer's four-chapter structure for a requirements document: Project, Environment, Goals, and System (Changes from Original SRS Meyer Template).
- **Perplexity** – A measure of how well a model predicts a body of text, where lower is better. Used to compare a split model with the unsplit one (Section S.8).
- **PIPEDA (Personal Information Protection and Electronic Documents Act)** – Canada's private-sector privacy law (Section G.7.2).
- **PoC (proof of concept)** – The first working demonstration of the system (Section P.3).
- **Prefill** – The first phase of a request, where the whole prompt runs through the model to fill the KV cache before the first output token (Section S.4).
- **Quantisation** – Converting a model's weights to a lower-precision format to reduce memory use (Section G.6).
- **RACI** – A chart showing who is Responsible, Accountable, Consulted, and Informed for each piece of work (Section G.4.3).
- **REST (representational state transfer)** – A style of HTTP API where clients read and change named resources (Section S.3).

- **RPC (remote procedure call)** – Calling a function on another machine as if it were local (Section G.2).
- **RTT (round-trip time)** – The time for a message to reach another node and for the reply to come back (FR.S.1.3).
- **Session** – A period of interaction with Lake in which one or more inference requests are served. For the operator, it runs from forming a cluster to shutting it down (UC1). For an inference user, it is one conversation whose context is kept between prompts (UC3).
- **SRS (Software Requirements Specification)** – This document.
- **Streaming** – Returning a response token by token as it is generated (FR.G.4.6).
- **TA (teaching assistant)** – The course staff member who advises the team and assesses its deliverables (Section G.7.1).
- **Telemetry** – Usage statistics, crash reports, update checks, or any other data that software sends to its developer or a third party. Lake sends none (NFR.S.11).
- **Tensor** – A multi-dimensional array of numbers. Weights, activations, and the KV cache are all tensors (Section S.1).
- **Tensor I/O** – The Lake component that compresses, sends, receives, and decompresses tensors between neighbouring nodes (Section S.1).
- **Token** – A unit of text processed by an LLM, typically a word or part of a word.
- **Token throughput** – The number of tokens generated per unit time, usually tokens per second.
- **Tool calling** – An OpenAI API feature where the model asks the client to run a named function and return the result (FR.G.4.6).
- **Transformer** – The neural network architecture of current LLMs, built from a stack of layers (Section G.6).
- **UI (user interface)** – The part of a program a person interacts with. Lake's UI is the web UI.

- **V&V (verification and validation)** – Verification checks that the system meets its requirements. Validation checks that it is the right system for its stakeholders (Section S.8).
- **Web UI** – The Lake component, served by the leader node and opened in a browser, where the operator forms a cluster, loads a model, and views performance (Section S.1).
- **Weights** – The learned parameters of a model that determine its behaviour. Lake never modifies them (NFR.E.4).
- **x86_64** – The 64-bit instruction set of Intel and AMD processors. One of the two CPU architectures Lake supports (NFR.S.14).

(G) Goals

This following explains why Lake exists and what it is for, in terms a reader outside the team can follow. How the system is built is left to the System chapter (Section E.7), and the world it runs in to the Environment chapter (Section G.8.0.3).

G.1 Context and Overall Objective

LLMs have become an everyday tool, and almost everyone who uses one does so through a commercial provider's web interface or application programming interface (API). The alternative is to run an open model on your own machine, which keeps your data at home but limits you to whatever fits in one device's memory. Most households, offices, and labs already own several computers, and most of them sit idle most of the time.

Lake runs a single pre-trained LLM across several consumer devices on one local network. The model is split by layer, each device holds and runs only its share, and the devices pass intermediate results between them so that together they answer a prompt as if the model were running on one machine. The objective is to let people run models that are too large for any one of their devices, privately and on hardware they already own, and to keep the model running as devices join and leave the network.

Figure 1 shows where Lake's boundary sits. Lake is the software on each participating device. The devices themselves, the network between them, the model file, and the tools a user sends prompts from are all outside the boundary.

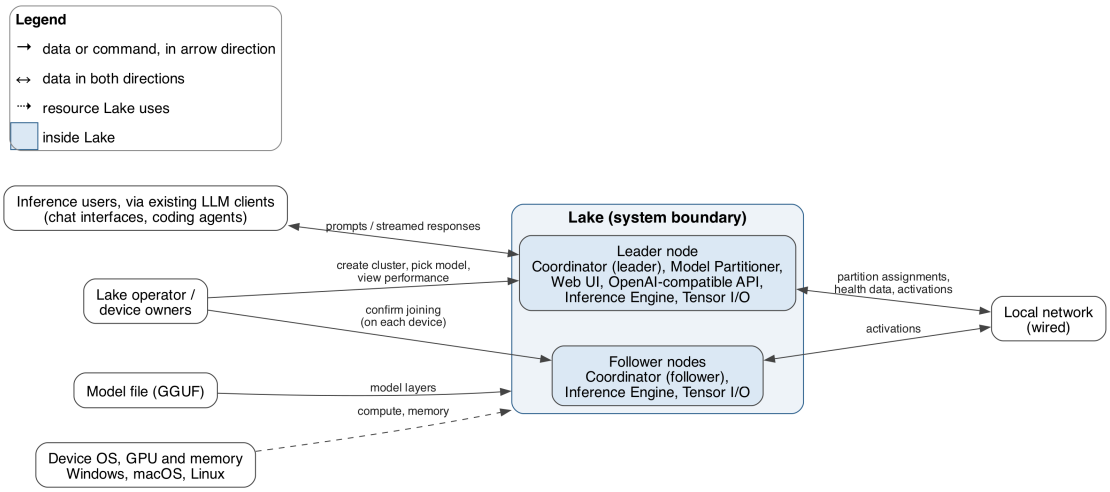


Figure 1: Context diagram showing Lake's boundary and the people, tools, files, and hardware around it.

G.2 Current Situation

Today there are two ways to use an LLM. The first is a hosted service from one of the major inference providers. These give the best model quality, but every prompt leaves the premises, the cost grows with use, and the user has no control over how their data is handled. For a clinic or a law office this is often not acceptable at all.

The second is local inference with a tool such as llama.cpp (**?**) or Ollama (**?**). The model runs in one process on one machine, so the largest model a user can run is set by the memory of their single best device. A laptop that runs a small model well may run a mid-sized one slowly and a large one not at all, even when three other computers sit on the same desk.

A few projects already spread a model over several machines: exo (**?**), prima.cpp (**?**), and the remote procedure call (RPC) backend of llama.cpp. They assign layers to devices once, when the model loads, and the assignment stays fixed for the session. If a device drops off the network the session fails, and a device added later goes unused until everything is restarted. Lake builds on the same idea but makes the assignment something the system can change while it is running. Table 3 summarises how Lake compares. Mixed operating systems are marked "partly" for the existing tools because exo runs on Apple silicon through MLX and only on the CPU on Linux, with no Windows build (**?**), and prima.cpp supports Linux and macOS (**?**); only the llama.cpp RPC backend runs on all three.

Table 3: Lake compared with the current options

	Hosted service	**Single-device local**	**Existing distributed tools**	**Lake**
Data stays on the user's network	No	Yes	Yes	Yes
Cost per use	Per token	None	None	None
Largest model	Provider's choice	One device's memory	Pooled memory	Pooled memory
Survives a device leaving	N/A	N/A	No, session fails	Yes, layers reassigned
Uses a device that joins later	N/A	N/A	No, needs restart	Yes
Mixed operating systems in one pool	N/A	N/A	Partly	Windows, macOS, Linux

G.3 Expected Benefits

The benefits below follow the five goals in the Problem Statement, as shown in the goal model in Figure 2. Each benefit states the measure we will use to check it. The requirements that deliver them are in Section S.2.

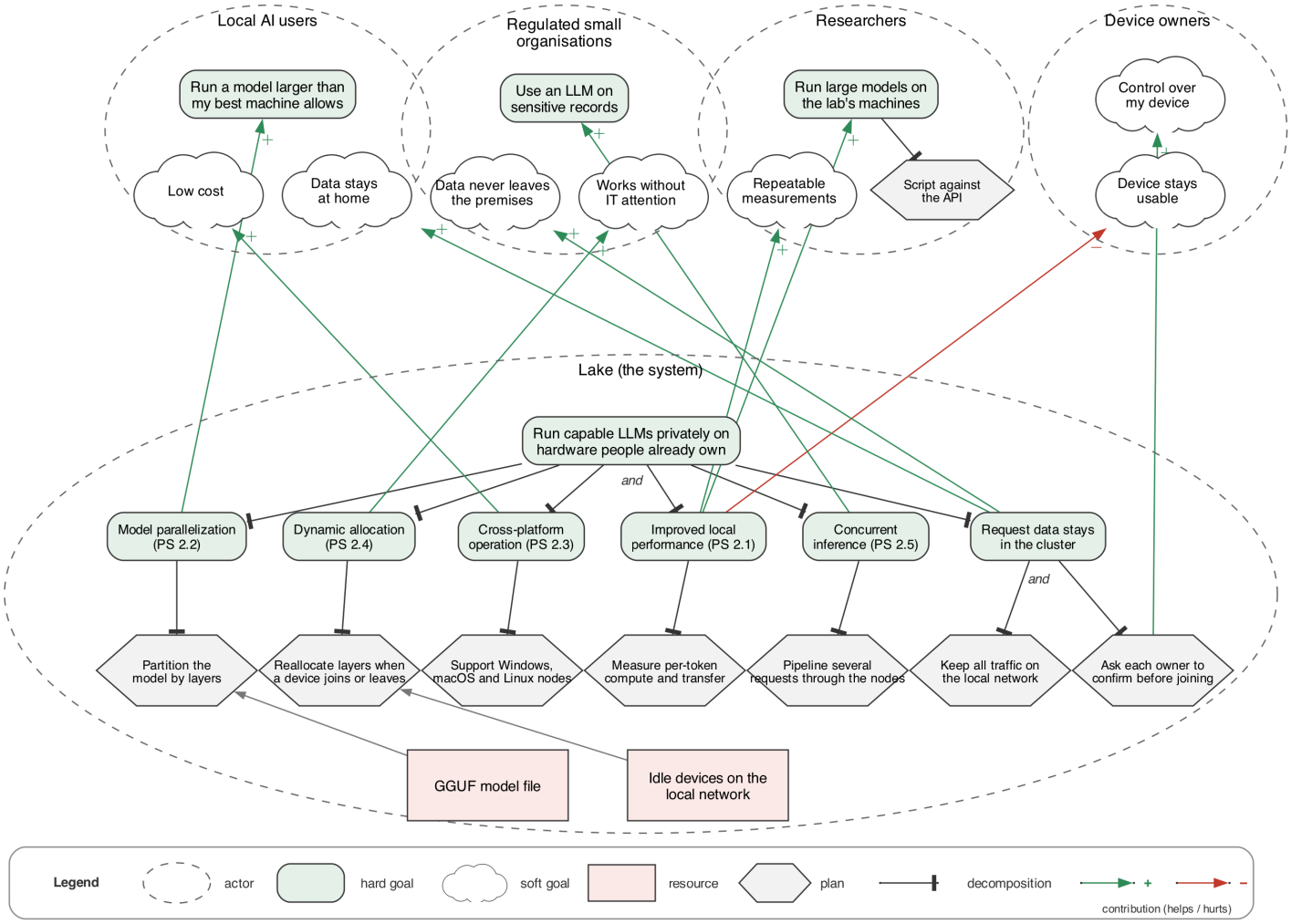


Figure 2: Goal model for Lake and its stakeholders.

Larger models on existing hardware

A user can run a model that does not fit on any one of their devices. The measure is the largest model, by parameter count, precision, and context window, that a cluster can serve compared with the largest the single best device in that cluster can serve alone. This is the primary benefit, and the Model Size requirement in Section S.2 states the target.

Privacy

Prompts, responses, and the model's intermediate data stay on the devices in the cluster and never go to an external provider. The measure is that no request data leaves the cluster during a session. What privacy does and does not cover in the first release is stated in Section G.6.

Lower cost

There are no per-token or subscription fees, and no new hardware is needed beyond what the user already owns. The measure is that a cluster can be formed entirely from machines the user had before installing Lake.

Resilience to devices coming and going

When a device leaves or joins, Lake reassigns layers and keeps serving requests as long as the remaining cluster has enough memory. The measures are whether requests are served after a reassignment and how long the reassignment takes, which Section S.9 defines precisely.

Mixed platforms in one cluster

Windows, macOS, and Linux machines can be combined, so a user is not limited to devices of one type. The measure is a working cluster that mixes all three.

Works with the tools people already use

Lake exposes the same API format as the major hosted providers, so existing chat interfaces and coding assistants can point at Lake instead of a cloud service with no changes on their side. The measure is an unmodified third-party client completing a session against Lake.

Several users at once

While one request's token is being computed on one node, another request's token can be computed on the next, so a cluster can serve several users without each one waiting for the others. The measure is the number of requests the cluster can serve at once before any single request slows by more than the tolerance set in the Scalability requirement in Section S.2, and how that number grows with the number of nodes.

Speed, where the split allows it

Response time and throughput (tokens per second) for a single request are secondary measures. A split model is not expected to beat a single device on a model that already fits on that device, since every token still passes through every layer in order. The gain is in capacity, and in throughput when several requests are served at once. The practitioner we interviewed (see Section G.7.2) expected the same: a split over an office network would likely be slower than one good GPU for live chat, but worth it for background work such as re-indexing documents, OCR, and overnight test runs, which is a use we had not considered.

G.4 Functionality Overview

This section is a capsule version of the System chapter. Each high-level requirement here maps to one or more of the components in Section S.1 (the Coordinator, the Model Partitioner, the Inference Engine, Tensor I/O, and the Web UI) and to detailed requirements in Section S.2.

G.4.1 Functional Requirements

These are the high-level requirements, stated for the reader of the Goals chapter. Each ends with the detailed, verifiable requirements in Section S.2 and the Environmental Requirements table that realise it, so a change in either place can be traced to the other.

- **FR.G.4.1 Form a cluster with consent.** Lake runs on each device. From the web user interface (UI) on one device, which becomes the leader node, the operator sees the other devices on the local network that are running Lake and chooses which to include. Each chosen device asks its owner to confirm before it joins, so no device takes part without consent. *Realised by* FR.S.1.4, FR.S.5.1, FR.S.5.2, FR.S.1.16, FR.S.1.17, FR.S.1.18, FR.S.1.20, FR.S.1.21, FR.S.5.7, and NFR.S.12.

- **FR.G.4.2 Load and partition a model.** The operator picks a model in GGUF format (**?**), the file format used by llama.cpp. Lake measures each node's memory, compute, and network performance, decides which range of layers each node will hold, and has each node load only that range. *Realised by* FR.S.5.3, FR.S.1.8, FR.S.2.1 to FR.S.2.4, FR.S.1.6, FR.S.1.19, FR.S.1.1, FR.S.3.1, NFR.S.4, and NFR.E.2.

- **FR.G.4.3 Run inference across the cluster.** A prompt enters at the first node, the activations pass from node to node in layer order, and generated tokens stream back to the user one at a time. Each node keeps the key-value (KV) cache for its own layers, so nothing about the conversation has to be re-sent between tokens. Several requests can be in flight at once, each at a different node, and their state is kept apart. *Realised by* FR.S.1.5, FR.S.1.15, FR.S.4.1, FR.S.4.3, FR.S.3.2, FR.S.3.3, FR.S.4.4, and, for concurrent requests, FR.S.1.14, FR.S.3.5, FR.S.4.6, and NFR.S.5.

- **FR.G.4.4 Reallocate when the cluster changes.** When a node leaves, fails, or joins, the leader re-partitions the model across the nodes that remain and resumes serving, retrying any request that was in progress. If the cluster no longer has enough memory for the model, the user is told why rather than left with a hung session. *Realised by* FR.S.1.10 to FR.S.1.13, FR.S.1.22, FR.S.2.2, and FR.S.5.4.

- **FR.G.4.5 Measure performance.** Lake records compute time on each node and transfer time between nodes. These measurements drive the partitioner and are shown in the web UI, which highlights the node or link that is the bottleneck and compares the cluster against the single best machine in it, so the operator can see whether pooling is paying off. *Realised by* FR.S.1.2, FR.S.1.3, FR.S.3.4, FR.S.4.5, FR.S.1.23, FR.S.1.7, FR.S.1.16, FR.S.5.5, FR.S.5.6, and NFR.S.1.

- **FR.G.4.6 Serve prompts through a standard API.** The leader node exposes an HTTP API in the OpenAI format (**?**), including streaming and tool calling, so any client that can talk to a hosted provider can talk to Lake. *Realised by* FR.S.1.9 and NFR.E.3.

Two of these carry most of the value: FR.G.4.2 and FR.G.4.4. Partitioning is what lets a model larger than any one device run at all, which is the primary goal of the project. Reallocation is what sets Lake apart from exo and the llama.cpp RPC backend, both of which fix the split when the model loads. Without the first

there is no product; without the second there is nothing a user could not already get elsewhere.

G.4.2 Non-Functional Requirements

These state how well Lake must do what Section G.4.1 describes. Each is a quality the system must have, not a design choice, and each names the detailed requirement that gives it a measurable target.

- **NFR.G.4.1 Request data stays in the cluster.** No prompt, response, or intermediate data is sent to any device outside the cluster, none is kept after a request completes, and Lake sends no telemetry. This is the privacy benefit, and it is the reason an organisation handling export-controlled or medical data could use Lake at all. *Realised by* NFR.S.10, NFR.S.11, NFR.S.12, and NFR.E.5.

- **NFR.G.4.2 Runs on Windows, macOS, and Linux.** Each is a supported platform for any node, and one cluster can mix all three. This is the cross-platform goal from the Problem Statement. *Realised by* NFR.S.3, NFR.S.13 to NFR.S.15, and NFR.E.1.

- **NFR.G.4.3 The split does not change the model.** A model run through Lake produces output of the same quality as the same model run on one machine, within the tolerance set by NFR.S.6. Users would not trust a system that made their model worse. *Realised by* NFR.S.6, FR.S.2.3, and NFR.E.4.

- **NFR.G.4.4 License compliance.** Lake respects the licenses of the software it builds on and of the models it runs, as set out in Section E.7. *Realised by* NFR.E.7 and NFR.E.8.

- **NFR.G.4.5 Installable without expert help.** A user can install Lake on each supported platform from an installer or a single package-manager command, without building from source or editing configuration files. Both the home user and the organisation in Section G.8 said they would not deploy a tool that needs more than that. *Realised by* NFR.S.3 and NFR.E.1.

- **NFR.G.4.6 Serves more users as nodes are added.** The number of requests a cluster can serve at once without any one of them slowing beyond a set tolerance grows with the number of nodes. This is the measure behind the concurrent inference goal. *Realised by* NFR.S.5.

Privacy and fidelity matter most. A local inference tool that leaked data or degraded the model would fail the two groups it is meant for: the privacy-bound organisation and the user who already has a model they like.

G.4.3 RACI Matrix

Table 4 shows who is responsible, accountable, consulted, and informed (RACI) for each high-level requirement, using the stakeholders from Section G.7.

Table 4: RACI matrix for the high-level requirements

Requirement	Local AI users	Regulated small orgs	Small AI labs	Device owners	Development team	Instructor and TA	Society
FR.G.4.1 Form a cluster with consent	C	C	C	C	R, A	I	–
FR.G.4.2 Load and partition a model	C	I	C	I	R, A	I	–
FR.G.4.3 Run inference across the cluster	C	I	C	I	R, A	I	–
FR.G.4.4 Reallocate when the cluster changes	C	C	C	C	R, A	I	–
FR.G.4.5 Measure performance	C	I	C	I	R, A	I	–
FR.G.4.6 Serve prompts through a standard API	C	I	C	–	R, A	I	–
NFR.G.4.1 Request data stays in the cluster	C	C	C	C	R, A	I	I
NFR.G.4.2 Runs on Windows, macOS, and Linux	C	C	C	C	R, A	I	–
NFR.G.4.3 The split does not change the model	C	I	C	–	R, A	I	I
NFR.G.4.4 License compliance	I	C	I	I	R, A	C	I
NFR.G.4.5 Installable without expert help	C	C	C	C	R, A	I	–
NFR.G.4.6 Serves more users as nodes are added	I	C	C	I	R, A	I	–

Table 5: RACI legend

Symbol	Definition
R	Responsible: who does the work
A	Accountable: who answers for it being done
C	Consulted: who gives input
I	Informed: who is kept up to date
–	No involvement

G.5 High-Level Usage Scenarios

These are the main ways people use Lake, stated in user terms without system detail or error cases. Figure 3 shows them as use cases with the three actors from Section G.8. Each use case is expanded, with special and erroneous cases, in Section S.6.

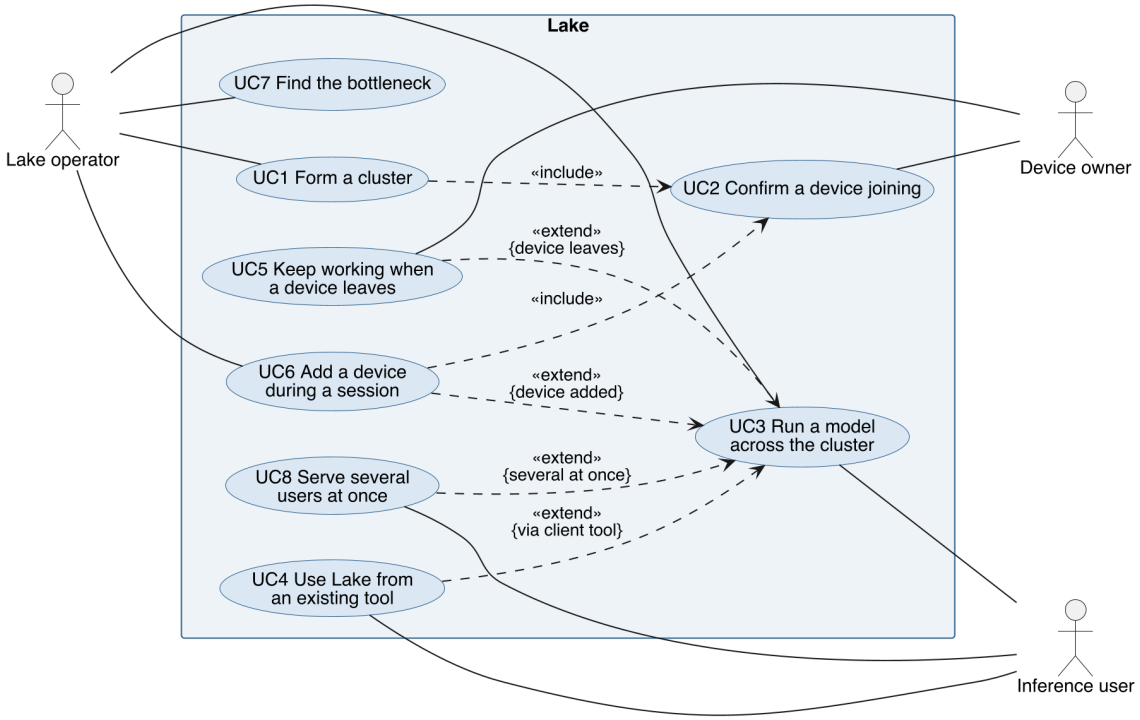


Figure 3: High-level use case diagram. Stick figures are actors, ellipses are use cases, and the box is Lake. A solid line joins an actor to a use case they take part in. A dashed arrow labelled *include* means the source use case always performs the target; one labelled *extend* means the source adds to the target under the condition in braces.

1. **UC1 Form a cluster**

   - The operator starts Lake on each computer they want to use.
   - On one of them, the operator opens Lake and sees the other computers on the network that are running it.
   - The operator selects the computers to include.
   - Each selected computer's owner confirms (UC2).
   - The cluster is shown with each computer's memory and platform.

2. **UC2 Confirm a device joining**

   - A prompt appears on the device asking whether it may join the named cluster.

- The owner accepts or declines.
- If accepted, the device joins and the owner can see what it is doing; if declined, it is left out.

3. **UC3 Run a model across the cluster**
   - The operator chooses a model file, which may be larger than any one computer can hold.
   - Lake reports that the model is loaded across the cluster.
   - A user sends a prompt.
   - The response streams back word by word.
   - The user sends further prompts in the same session.
4. **UC4 Use Lake from an existing tool**
   - A user opens the chat interface or coding assistant they already use.
   - They point it at Lake's address instead of a cloud provider.
   - They use the tool as before; prompts and responses go through Lake and stay on the local network.
5. **UC5 Keep working when a device leaves**
   - During a session, a computer in the cluster is closed or disconnected by its owner.
   - Lake tells the operator which device left and that it is reorganising.
   - The user's current prompt is answered, or re-run, without being resubmitted. A partial answer is never presented as a complete one.
   - If the remaining computers cannot hold the model, the operator is told why and what would fix it.
6. **UC6 Add a device during a session**
   - The operator selects a new computer that has appeared on the network.
   - Its owner confirms (UC2).
   - Lake reorganises to use the new computer and shows the new arrangement.
   - The user continues prompting and sees the change in speed.

7. **UC7 Find the bottleneck**

    - Responses feel slow, so the operator opens the cluster view.
    - The view shows how long each computer and each connection takes per word generated, and marks the slowest.
    - The operator decides whether to swap a cable, remove a computer, or add one.

8. **UC8 Serve several users at once**

    - Two or more users send prompts to the same cluster at the same time, from their own tools.
    - Each sees their own response stream back, and none sees another's prompt or answer.
    - Each response arrives about as fast as it would have alone, until the cluster is busy enough that the operator sees requests queue in the cluster view.

G.6 Limitations and Exclusions

Lake is deliberately narrow. The following are out of scope. The stretch goals from the Problem Statement are listed as exclusions until the team decides to take them on. The likely and unlikely changes listed at the end of the System chapter record which of these may change.

- *No training or fine-tuning.* Lake runs pre-trained transformer models that ggml (**?**) supports. Lake does not train, fine-tune, quantise, or otherwise modify a model, and does not change the quality of its output.

- *Trusted network and devices.* In the first release, traffic between nodes is not encrypted and devices are not authenticated beyond the owner's confirmation. Privacy therefore means that data never leaves the user's own network, not that it is protected from other devices on that network. Users must run Lake only on devices they trust and a network they control. Encryption and authentication are a stretch goal.

- *Supported hardware.* Each device needs a GPU from Nvidia, AMD, or Apple, because inference on a CPU alone is too slow to be useful in a cluster. Smartphones are a stretch goal.

- *Wired connections first.* The first revision targets wired links: Thunderbolt for the two-device proof of concept and Ethernet for larger clusters. Wireless operation is a stretch goal because its latency and variability may make the split model too slow.
- *Not faster for models that already fit.* Lake does not speed up a model that already runs well on one device. The benefit is running models that would not otherwise fit.
- *No model hosting.* Lake does not download or distribute model weights. Users supply a model file under that model's own licence.

G.7 Stakeholders and Requirements Sources

G.7.1 Stakeholder Definition

Table 6 lists the categories of people who can affect Lake or be affected by it. Direct stakeholders use or build the system; indirect ones are affected by it without touching it. The three direct user groups are developed into personas in Section G.8.

Table 6: Stakeholders

Stakeholder	Type	Description	Justification
Local AI users	Direct	People who already run models on their own machines with tools like llama.cpp or Ollama and want to run larger ones.	The primary hands-on users, and the group the team itself belongs to. Their habits set the baseline Lake has to beat.

Table 6 continued

Stakeholder	Type	Description	Justification
Organisations in regulated industries	Direct	An engineering firm with export-controlled data, a medical clinic, or a law office that wants AI assistance on sensitive records but cannot let the data leave the building. The one we interviewed already runs an in-house assistant for about two hundred staff on its own GPUs.	For them privacy is a legal obligation, not a preference. They are the clearest case for running inference on premises, the reason the privacy benefit has to be exact about what it covers, and the source of the deployment needs in Section G.8: managed machines only, no telemetry, and an off switch.
Small AI labs and research groups	Direct	Groups with a few machines and a modest budget who want to run large models without renting cloud GPUs.	They push the model-size and measurement requirements hardest and are the most likely to use the API directly.

Table 6 continued

Stakeholder	Type	Description	Justification
Device owners	Direct	People whose computer joins a cluster, who may not be the person running it.	Their device gives up memory, compute, battery, and bandwidth while it takes part. They must consent, be able to see what their device is doing, and be able to pause it; the interviewee's first condition for pooling office machines was that Lake back off the moment the owner is working.
Development team	Direct	The five students building Lake as a capstone project.	Responsible for every requirement here and bound by the course timeline, which limits scope.
Course instructor and teaching assistant	Indirect	Assess the project against the course rubrics and give feedback on each deliverable.	Their feedback changes the requirements, and the deliverable schedule in Section P.3 is theirs.
Open-source projects Lake builds on	Indirect	ggml, llama.cpp, and the GGUF format, with their maintainers and licenses.	Their interfaces decide which models and hardware Lake can support, and their licenses constrain how Lake is distributed.

Table 6 continued

Stakeholder	Type	Description	Justification
Society	Indirect	The public, and in particular people and organisations who cannot pay for hosted inference or a high-end machine.	Lake lowers the cost of using capable models privately, which broadens who can use them. Health data is the clearest case. The same applies outside wealthy, English-speaking markets: a school or small business anywhere with a few ordinary computers can run an open model in its own language. Lake also raises energy use on participating devices, and a model run through Lake carries whatever biases and risks it had before, since Lake does not alter it. The Hazard Analysis covers the safety side.

G.7.2 Requirements Sources

Beyond the stakeholders above, the requirements in this document come from the following sources.

G.7.2.1 Project documents. The Problem Statement and Development Plan set the goals and their metrics, the stretch goals, the hardware limits, and the technology direction. The Hazard Analysis contributes the safety and security requirements

and the critical assumptions.

G.7.2.2 Peer review. Team 07's review of the Problem Statement filed six issues in the project repository. Two of them clarified what privacy covers in the first release and made dynamic allocation measurable.

G.7.2.3 Teaching assistant. Meetings with the teaching assistant recommended this template and shaped how the requirements are organised.

G.7.2.4 User interview. On October 16, 2026 we sent the eight questions in Appendix 1 to an AI developer at an aerospace engineering firm, who answered in writing. He builds and runs the company's in-house assistant for about two hundred staff, entirely on two GPU machines, because much of the firm's data is export-controlled and cannot go to a cloud provider. The interview confirmed the privacy benefit as the reason such organisations run locally at all, confirmed the OpenAI-compatible API as a must (with streaming, tool calling, and structured output), and confirmed the bottleneck view in UC7. It corrected us in three places: a split over an office network is expected to be slower than one good GPU for live chat, so the honest pitch is capacity and background work rather than speed; a retried request must never repeat side effects or present a cut-off answer as complete (UC5); and a deployment in an organisation needs managed machines only, no telemetry, no automatic updates, and a way for each owner to pause their machine, which shaped the second persona and the device-owner stakeholder.

G.7.2.5 Regulation and guidance. No law requires an organisation to run its models locally, but several make it the simplest way to comply. Canada's private-sector privacy law, PIPEDA (**?**), keeps an organisation accountable for personal information it passes to a third party. Export control regimes such as Canada's Controlled Goods Program under the Defence Production Act (**??**) restrict who may examine or possess controlled technical data, which includes software and documentation, and this is the kind of rule that keeps the interviewee's firm off hosted services. NIST's generative AI profile (**?**) lists data privacy, information integrity, and information security among the risks an operator of a generative model is expected to manage. These documents are the background for NFR.G.4.1 and the no-telemetry expectation in Section G.8. Which of them bind a given deployment depends on who deploys Lake; the standards and licences that bind Lake itself are in Section E.7.

G.7.2.6 The team as users.

The team's own experience running models locally is the source of the first persona and of the use cases in Section G.5.

G.7.2.7 Existing distributed inference tools.

exo (**?**), prima.cpp (**?**), and the RPC backend of llama.cpp (**?**) were studied to learn what current systems do and where they fall short. The gap they share, a split that is fixed when the model loads, is the origin of the dynamic allocation goal.

G.7.2.8 Libraries and formats.

The ggml and llama.cpp documentation and source (**???**) define which models and hardware can be supported and how a model can be split. The OpenAI API reference (**?**) defines the request and response format Lake's API follows.

G.7.2.9 Course material and the template.

The course templates, rubrics, and lectures, and Meyer's handbook (**?**), which this document's structure follows.

G.8 User Profiles

The Environment chapter distinguishes three roles: the inference user who sends prompts, the Lake operator who sets up and runs the cluster, and the device owner who contributes a machine. The personas below represent the three direct user groups from Table 6 and show how those roles land on real people. They are composites, not individuals.

G.8.0.1 Local AI user *Priya Nair, 29, software developer*

"I've got a desktop with a good GPU and two laptops that mostly sit there. I want to run the 70B model, not the 8B one."

Priya runs models on her gaming desktop with Ollama and llama.cpp and knows what quantisation and context length mean. She reads release notes, has opinions about which models are good, and is comfortable with a command line, installing software on several operating systems, and reading logs. Her problem is that the model she wants does not fit on her desktop, and a bigger GPU costs more than she will spend on a hobby. She is the operator, the device owner, and the inference user all at once. She will tolerate rough edges if the system works, which makes her the right first user, and she drives the requirements for straightforward installation on each platform, GGUF model support, and clear reporting of what each device loaded.

G.8.0.2 Operator in a regulated organisation *Nadia Reyes, 34, AI developer at an engineering firm*

"We removed the cloud provider from our pipeline. Export-controlled documents can't leave the building, so everything runs on our own GPUs."

Nadia is based on the practitioner we interviewed. She builds and runs an in-house assistant used by two hundred colleagues, serving a 27-billion parameter model from two workstation GPUs, and she has already tuned prompt caching and speculative decoding to get the speed she needs. She knows inference well, reads vLLM metrics daily, and tests every release before it reaches users. She is the operator; her colleagues are the inference users through a chat interface and office add-ins; the company owns the devices. What stops her running a larger model is memory per card, and her interest in Lake is pooling idle workstations for background jobs such as re-indexing, OCR, and overnight evaluation, not live chat. She would deploy it only with signed installers pushed by IT, no telemetry, no automatic updates, company-managed machines only, and a pause switch on every node. She drives the requirements for the OpenAI-compatible API with streaming and tool calling, for comparing the cluster against the single best machine, for exportable metrics, for the no-telemetry privacy guarantee, and for retries that never repeat side effects.

G.8.0.3 Researcher *Dr. Mei Lin, 35, postdoctoral researcher leading a four-person NLP group*

"We have six workstations and no cloud budget. I need numbers I can put in a paper."

Mei understands transformer models and inference and wants to compare models and settings. She will use the API directly and script against it, and she cares about repeatable measurements of speed and overhead as much as about the model running at all. She is usually both the operator and an inference user, and her lab mates are the device owners. She drives the requirements for the OpenAI-format API, the per-token performance measurements, and for comparing a split run against a single-device run.

(E) Environment

E.1 Glossary

The glossary has been moved to the front of the document (see the Glossary and the Changes from Original SRS Meyer Template).

E.2 Environment Components

- **Inference users (a.k.a users)** – Individuals who interact with the Lake network to serve inference requests.
- **Lake operators** – Individuals who manage the Lake network, including the device pool and the model.
- **Device owners** – Individuals who contribute one or more nodes towards the device pool.
- **Operating systems and drivers** – Windows, macOS, and Linux operating systems that have the supported drivers to facilitate inference requests. Android and iOS operating system support is a stretch goal.
- **Network equipment** – Routers, switches, Thunderbolt cables, and any other hardware necessary to connect the nodes to the network. Connectivity via Wi-Fi is a stretch goal.
- **Model** – Pre-trained and open-source model weights that are permitted to be downloaded and utilised by the Lake network via model layer partitioning.
- **External API consumers** – Other services that utilise the Lake API to serve inference requests. This acts as the main interface for consumers to use Lake, such as coding agent harnesses (e.g. Claude Code, Codex, Pi).

E.3 Constraints

- **Device hardware** – Each node should have a sufficient CPU, GPU, memory, and storage such that its assigned model layers, KV cache, and other Lake software components can fit and run within the device, such that distributed inference has both a higher token throughput and lower latency than an equivalent single-device inference.

- **Network communication** – Network communication, whether wired or wireless, should have a sufficiently high bandwidth and low latency. Inter-node communication overhead may make distributed inference slower than single-device inference.
- **Platform compatibility** – Nodes within the Lake network must operate on Windows, macOS, and Linux (as well as Android and iOS as stretch goals) as hardware acceleration and inference platforms are limited by the available drivers on these operating systems.
- **API compatibility** – Lake must implement a standardised OpenAI API spec such that existing external clients can easily utilise Lake, (**?**), as specified in Section S.3.
- **Model architecture** – Only supported LLM architectures that Lake's model layer partitioning supports can be used.

E.4 Assumptions

- **Device owners authorise their nodes for use** – Device owners have authorised their device to be used in the Lake network as a node for serving inference.
- **Participating nodes are trusted devices within the Lake network** – Each node is trusted to run and process potentially sensitive inference data.
- **Permitted model use** – Lake operators are using permitted models for inference that adhere to the respective licensing.
- **Network reachability** – Nodes within the Lake network must be able to communicate over a network connection.
- **Model suitability** – Models that are suited for a given task are used when Lake is serving inference (e.g. a larger model for logic/coding-based tasks, a smaller model for conversational tasks).
- **Trusted local network** – The local network connecting the participating nodes is private and trusted by the operator.

E.5 Effects

- **Resource usage** – Device owners' CPU/GPU, memory, and disk usage will increase while the device is part of the Lake network, potentially making the device too resource constrained to multi-task or perform other activities. When not in use, any cached data or the stored layers can continue to utilise disk space.
- **Energy usage** – Devices will consume more power, increasing energy usage, heat generated, fan usage, and decreasing battery life if unplugged.
- **Network bandwidth usage** – The Lake network may utilize a large amount of a network's bandwidth, inhibiting other devices or processes from accessing the network.
- **Logged data** – During inference, logs and other diagnostic information can be stored and persist on the node's disk. Logs must exclude sensitive request data as specified in SR5-2.
- **Access to inference** – Users can gain access to inference that they previously would not be able to get within a single device or with a paid service.

E.6 Invariants

- The provided model must remain unchanged. Loading the model and partitioning its layers must not mutate the underlying model weights.
- Inference must be contained within the Lake network, and must not be sent to 3rd-party services.
- All nodes within the Lake network must be using compatible layer partitions that originate from the same model's layers.
- All nodes must serve inference in a way that the activations are performed in the same order as in single-device inference.
- Intermediate data from one request must not be used for another independent request.

E.7 Standards and Legal/Regulatory Factors

Standards:

- **API compatibility** – Lake's inference API will follow the OpenAI API endpoints for requests and response formats, as specified in S.3.
- **Coding conventions** – Black code style and Ruff lint checks will be used for Python components (**??**). Rustfmt and Clippy will be used for Rust components (**??**). Prettier and ESLint will be used for TypeScript components (**??**).

Legal and Regulatory Factors:

- **Software licensing** – Lake is available under the GNU GPL v3.0 licence (**?**). All dependencies used in Lake must be compliant and compatible with Lake's intended use and distribution.
- **Model licensing** – Pretrained LLMs used for inference are subject to their respective licenses. Lake must be compliant with these licenses for its use and distribution.

Environmental Requirements

Table 7: Environmental Requirements

Label	Requirement
NFR.E.1	The system shall be installable and usable on the supported platforms of Windows, macOS, and Linux, and Android/iOS as a stretch goal. Related to NFR.S.3 and NFR.S.13.
NFR.E.2	The system shall perform model layer partitioning to distribute model inference across various supported devices. Related to SR1-1 and SR1-2.
NFR.E.3	The system shall follow the OpenAI API specification for its request and response API service. Related to FR.S.1.9.
NFR.E.4	The system shall not modify the underlying model weights during model layer partitioning.
NFR.E.5	The system shall contain all inference requests within the Lake network and not send any requests to 3rd-party services. Related to NFR.S.11.
NFR.E.6	The system shall adhere to all codebase formatting and linting conventions.
NFR.E.7	The system shall follow the GNU GPL v3.0 license in applicable use cases and distributions.
NFR.E.8	The system shall adhere to a pretrained model's licensing requirements for use and distribution within Lake.

(S) System

S.1 Components

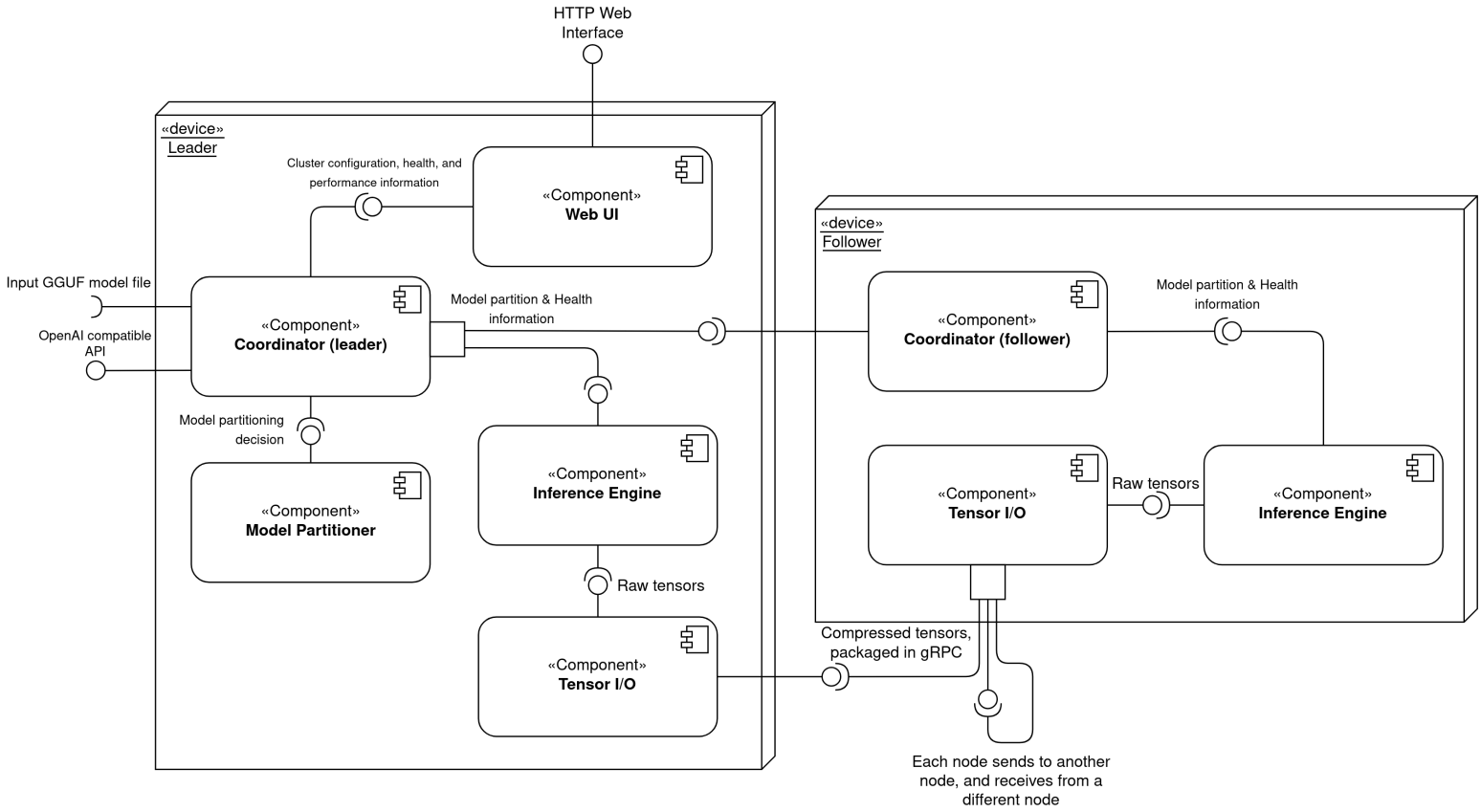


Figure 4: Component diagram.

- **Coordinator** – This component exists on all nodes, but the following responsibilities are only run on the leader node:
  - Providing an OpenAI-API compatible API to the user for use with tools like OpenCode (**?**).
  - Tracking cluster health, and detecting node failures.
  - Notifying nodes of partitioning changes.

  - Rebuilding the cluster and saving in-progress requests in the case of node failure, removal, or addition.
  - Measuring node connection performance.

  On a follower node, it is primarily responsible for managing the inference engine on its device, following the instructions of the leader node.

- **Model Partitioner** – This component determines how to partition the model among the participating devices. It decides at which points to cut the graph, how large each partition is, and which device gets each partition. This component is integral to ensuring that each device is assigned an appropriate partition given the device's computational and networking performance.

- **Inference Engine** – This component is the high-performance inference engine running on each device. It will handle execution of the given model partition for each device. It will call into ggml (**?**) using the Rust-to-C FFI to control the execution of a device's model partition.

- **Tensor I/O** – This component handles the compressing, sending, receiving, and decompressing of tensors between devices. It will leverage gRPC (**?**) for high-performance communication, with the potential to use a custom solution should gRPC fail to meet our requirements in practice.

- **Web UI** – This component handles displaying cluster information and structure to the user, as well as allowing user inputs to configure a new cluster, or modify an existing cluster.

S.2 Functionality

S.2.1 Coordinator

Functional Requirements

- **FR.S.1.1** The system shall provide the local Inference Engine with the model partition assigned to the local device.

- **FR.S.1.2** The system shall collect device health/performance information[1] from the local Inference Engine on a *HEALTH_INTERVAL* second interval. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.

[1] e.g. token throughput, end-to-end partition latency, backpressure.

- **FR.S.1.3** The system shall collect the network health/performance information[2] for established connections from the Tensor I/O component on a *HEALTH_INTERVAL* second interval. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.
- **FR.S.1.4** The system shall continuously scan for all other devices on the same network running Lake.
- **FR.S.1.5** The system shall provide the upstream and downstream node info to the local Tensor I/O.
- **When running on a leader node:**
  - **FR.S.1.6** The system shall send model partitions to other Coordinators on their respective devices as determined by the Model Partitioner. Partially satisfies hazard H0-2 from the Hazard Analysis.
  - **FR.S.1.7** The system shall monitor cluster performance by gathering performance data from each follower node's Coordinator. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.
  - **FR.S.1.8** The system shall provide the Model Partitioner with the available nodes, and gathered node performance data, to help inform optimal model cut points.
  - **FR.S.1.9** The system shall provide an HTTP API matching OpenAI's API.
  - **FR.S.1.10** The system shall detect failing follower nodes[3]. Partially satisfies hazards H2-1, H4-2 and H4-1 from the Hazard Analysis.
  - **FR.S.1.11** The system shall rebuild the cluster on a node failing or leaving. Partially satisfies hazards H2-1, H4-2 and H4-1 from the Hazard Analysis.
  - **FR.S.1.12** The system shall rebuild the cluster on a node joining.
  - **FR.S.1.13** The system shall retry in-progress requests after a cluster rebuild.
  - **FR.S.1.14** The system shall accept multiple concurrent requests through the HTTP API. Related to NFR.S.5.

[2] e.g. RTT, throughput, backpressure.

[3] A "failing" device is defined as severely degraded (*FAIL_THRESHOLD* of historical) token throughput, whether it be due to network or device resource instability.

- **FR.S.1.15** The system shall communicate the cluster graph structure to each follower.
- **FR.S.1.16** The system shall provide the Web UI with cluster information, and accept cluster configuration instructions from the Web UI.
- **FR.S.1.17** The system shall not begin inference until all selected followers have confirmed participation. Satisfies NFR.S.12.
- **FR.S.1.18** The system shall only include selected followers in a new cluster. Satisfies NFR.S.12.

- **When running on a follower node:**
  - **FR.S.1.19** The system shall accept model partition assignments from the leader node Coordinator.
  - **FR.S.1.20** The system shall, upon receiving an invite to join a cluster, request that the user confirm or decline participation. Satisfies NFR.S.12.
  - **FR.S.1.21** The system shall not partake in a cluster without user confirmation. Satisfies NFR.S.12.
  - **FR.S.1.22** The system shall not require confirmation to re-join a rebuilding cluster led by the same device, so long as the local device was already participating in the original cluster.
  - **FR.S.1.23** The system shall report collected health/performance data to the leader node Coordinator on a *HEALTH_INTERVAL* second interval. Partially satisfies hazards H0-1 and H0-2 from the Hazard Analysis.
  - **FR.S.1.24** The system shall assume the leader node has failed if there has been no health query from the leader node for more than *LEADER_TIMEOUT* consecutive seconds.
  - **FR.S.1.25** The system shall initiate the shutdown of the local node, if the leader node is assumed to have failed. Partially satisfies hazards H4-2 and H4-1 from the Hazard Analysis.

    Rationale: The Leader node is the device the user is physically using to interact with the cluster, so failure of that device will remove the user's ability to use the cluster, whether or not the cluster survives. The architecture of Lake will allow the ability to recover from a leader failure to be implemented at a later date.

S.2.2 Model Partitioner

Functional Requirements

- **FR.S.2.1** The system shall assign all devices a partition smaller than their available GPU-accessible memory, with additional *MEMORY_MARGIN* left for runtime overhead. Related to NFR.S.4. Satisfies hazard H1-3 from the Hazard Analysis.
- **FR.S.2.2** The system shall report failure in the case that no valid partition across the cluster exists (e.g. not enough GPU-accessible memory).
- **FR.S.2.3** The system shall produce partitions that when assembled, are equivalent to the original model. Satisfies NFR.S.6.
- **FR.S.2.4** The system shall assign larger partitions to more capable devices[4]. Related to NFR.S.4.

S.2.3 Inference Engine

Functional Requirements

- **FR.S.3.1** The system shall load the model partition given by the local Coordinator into GPU-accessible memory.
- **FR.S.3.2** The system shall accept incoming tensors from Tensor I/O, and run them through the loaded partition.
- **FR.S.3.3** The system shall send output tensors to Tensor I/O.
- **FR.S.3.4** The system shall measure partition performance metrics[5] and provide them to the local Coordinator on a *HEALTH_INTERVAL* second interval.
- **FR.S.3.5** The system shall support processing tensors from multiple different requests concurrently. Related to NFR.S.5.

[4]The "capabilities" of any given devices are defined via a combination of network throughput, GPU TFLOPs/TOPs, GPU-accessible memory size, and GPU-accessible memory bandwidth (GB/s).

[5]e.g. token throughput, end-to-end partition latency, backpressure.

S.2.4 Tensor I/O

Functional Requirements

- **FR.S.4.1** The system shall establish connections with neighbouring nodes' Tensor I/O components.
- **FR.S.4.2** The system shall dynamically choose the compression level of outgoing tensors to maximise throughput based on available CPU-time and network throughput.
- **FR.S.4.3** The system shall accept input tensors from the upstream node, decompress them, and provide them to the Inference Engine.
- **FR.S.4.4** The system shall accept output tensors from the Inference Engine, compress them, and send them to the downstream node's Tensor I/O.
- **FR.S.4.5** The system shall measure network performance data[6], for established connections, and provide that data to the local Coordinator on a *HEALTH_INTERVAL* second interval.
- **FR.S.4.6** The system shall maintain request association when sending tensors[7]. Related to NFR.S.5.

S.2.5 Web UI

Functional Requirements

- **FR.S.5.1** The system shall allow a user to scan for other connected devices running Lake. Partially satisfies NFR.S.2.
- **FR.S.5.2** The system shall allow a user to select connected devices to create a cluster. Partially satisfies NFR.S.2.
- **FR.S.5.3** The system shall allow a user to select an input GGUF file (**?**). Partially satisfies NFR.S.2.
- **FR.S.5.4** The system shall display an error to the user if partitioning the selected input file is infeasible given the selected devices. Partially satisfies NFR.S.1.

[6] e.g. RTT, throughput, backpressure.

[7] This is to allow tensors from different concurrent requests to be differentiated.

- **FR.S.5.5** The system shall display the performance metrics of the cluster as a whole, as well as for each individual node and connection. Partially satisfies NFR.S.1.

- **FR.S.5.6** The system shall highlight nodes or network connections causing bottlenecks in the cluster. Partially satisfies NFR.S.1. Satisfies hazard H4-3 from the Hazard Analysis.

- **FR.S.5.7** The system shall, upon receiving an invite to join a cluster, display a prompt for the user to either confirm or decline the invitation. Satisfies NFR.S.12.

S.2.6 Non-Functional Requirements

S.2.6.1 Usability

- **NFR.S.1 Understandability:** The user must be able to easily understand cluster information. A new user must be able to describe which devices are participating, the token throughput of the system, and which devices/connections are bottlenecks within *UI_TASK_TIME* of seeing the interface. Partially satisfied by FR.S.5.4, FR.S.5.5, and FR.S.5.6.

- **NFR.S.2 Controllability:** The user must be able to easily control cluster configuration. A new user must be able to create a new cluster within *UI_TASK_TIME* of seeing the interface. Partially satisfied by FR.S.5.1, FR.S.5.2, and FR.S.5.3.

- **NFR.S.3 Installability:** The user must be able to easily install Lake natively on Windows, macOS, and Linux. On Windows and macOS, the installation process should be automated and should require at most *INSTALL_CLICKS* clicks after downloading the installer. On Linux, the installation process should only require a single package manager command.

S.2.6.2 Performance

- **NFR.S.4 Model Size:** The cluster must be able to run a model with more parameters, higher precision, or larger context window than would be able to run individually on the node with the most memory. Related to FR.S.2.1, and FR.S.2.4.

- **NFR.S.5 Scalability:** The number of concurrent requests with $< MAX_DEGRADATION$ individual request performance degradation shall scale linearly with the number of participating devices. Related to FR.S.1.14, FR.S.3.5, and FR.S.4.6.
- **NFR.S.6 Intelligence:** The partitioned model must score at least *BENCH_RATIO* as well as the same model running on one machine using relevant benchmarks[8]. Satisfied by FR.S.2.3.

S.2.6.3 Look and Feel

- **NFR.S.7 Consistency:** Colours and fonts must be used consistently across the application.

S.2.6.4 Fault-Tolerance

- **NFR.S.8** The system shall reject invalid or corrupted GGUF files (**?**), and notify the user of the issue.
- **NFR.S.9** The system shall produce correct results in the face of network packet corruption and dropping up to *MAX_LOSS* packet loss. Partially satisfies hazard H3-1 from the Hazard Analysis.

S.2.6.5 Privacy

- **NFR.S.10** The system shall not save any request data[9] after the request has completed.
- **NFR.S.11** The system shall not send any data outside the cluster, except responses to the client and discovery and invitation messages to devices on the local network. This includes request data, usage statistics, crash and diagnostic reports, update checks and any other telemetry. Health and performance reports between nodes are allowed. **Rationale:** The developer we interviewed works with export-controlled data and needs Lake to run offline with no telemetry or automatic updates. Related to NFR.G.4.1, FR.S.1.4, FR.S.1.20, FR.S.1.23 and FR.S.5.1.

[8] Relevant benchmarks are entirely dependent on what types of model are being tested. Example benchmarks: SWE-bench (**?**) and FrontierMath (**?**).

[9] **Request data** refers to any data sent to Lake as part of the request, or any data that would give insight into what the data sent in the request might have been.

- **NFR.S.12** The system shall not include untrusted devices in a cluster. Satisfied by FR.S.1.17, FR.S.1.18, FR.S.1.20, FR.S.1.21, and FR.S.5.7. Satisfies hazard H5-1 from the Hazard Analysis.

S.2.6.6 Compatibility

- **NFR.S.13 Operating System:** The system shall be compatible with Windows, macOS, Linux.
- **NFR.S.14 CPU Architecture:** The system shall be compatible with x86_64 and ARM CPU architectures.
- **NFR.S.15 GPU Technology:** The system shall be compatible with NVIDIA, AMD, and Apple GPUs.

S.3 Interfaces

Lake requires the following interfaces:

- **OpenAI compatible API** – An HTTP server, following the OpenAI API structure. This will be the API used to interact with the user, via third-party tools of the user's choice that accept the OpenAI API, such as OpenCode (**?**).
- **GGUF input model** – An input file using the GGUF file format (**?**). This contains the model Lake will partition and run.
- **Rust-to-C FFI** – The Rust-to-C foreign function interface will be used to call ggml (**?**) from the Inference Engine.
- **gRPC between nodes** – A gRPC-based API used to communicate between nodes.
- **REST API between Coordinator and Web UI** – A REST API for communication between the leader Coordinator and the Web UI.

S.4 Normal Operation

1. User starts the Lake client on all participating machines.
2. User creates a Lake cluster using the UI on one of the devices, and provides a GGUF file (**?**). This device becomes the leader node.

3. Leader node checks connectivity with follower nodes, and collects performance metrics for each node.
4. Partitioner devises a partitioning scheme based on the collected metrics.
5. Leader node sends the model partitions to their respective devices.
6. Neighbouring nodes on the partition graph establish direct peer-to-peer (P2P) connections.
7. User initiates inference request by providing a prompt via the OpenAI compatible API.
8. Leader node sends the prompt tokens to the inference engine corresponding to the first node of the graph, initiating prefill.
9. Prompt tokens propagate through the graph until prefill completes, and the first output token is generated.
10. Output tokens are repeatedly fed back into the top of the graph, generating a new token on each iteration until the response is complete.
11. As the response generates, the user receives the generated output via the OpenAI compatible API.
12. User initiates a shutdown of the Lake cluster.
13. Each node unloads its portion of the graph.
14. Leader node saves the cluster configuration to disk.
15. All nodes sever their connections, and every Lake client terminates.

S.5 Undesired Event Handling

- **Model is too large for the available memory in the cluster** – The Partitioner reports a partitioning failure, and this error is communicated to the user via the Web UI.
- **Follower node fails during inference** – Leader node re-tries inference, if it continues to fail, the leader rebuilds the cluster, excluding the problematic device. Associated with hazard H2-1 in the Hazard Analysis.

- **Tensors are corrupted on transfer between nodes** – The receiving Tensor I/O requests that the tensor is retransmitted from the source Tensor I/O. Associated with hazard H3-1 in the Hazard Analysis.
- **Connection between two nodes is lost** – The nodes in question attempt to re-establish a connection. If this fails, the leader must rebuild the cluster, excluding the failed link. Associated with hazard H4-1 in the Hazard Analysis.
- **Input GGUF file is corrupted or fails to load** – Refuse to begin inference, and inform the user of the corrupted or faulty file.
- **A request is made with a larger prompt than the cluster has memory for** – If there are existing requests running, wait for them to complete, then try again. If there is still not enough memory, return an error for the request, and instruct the user to shorten their prompt or context window.
- **A rebuilt cluster is too small to hold the model after a node leaves** – The Partitioner reports a partitioning failure, and this error is communicated to the user via the Web UI.
- **A user attempts to create a cluster with a node without a GPU** – The Web UI displays an error message outlining the requirement for all nodes to have a GPU.

S.6 Detailed Usage Scenarios

This section expands each high-level scenario from Section G.5 (UC1–UC8) into a full use case with preconditions, guarantees, a numbered main success scenario, and extensions. Step numbers in the extensions refer to the main success scenario of the same use case.

UC1: Form a cluster

Use Case Name: Form a cluster
Primary Actor: Operator
Stakeholders and Interests:

- Operator: wants to pool the available computers into one cluster with little effort and to see what the cluster contains.
- Device Owner: wants no device to take part without the owner's consent.

- Inference User: wants a working cluster to send prompts to.

Preconditions:

- The Lake client is installed on every computer the operator wants to use.
- The computers are on the same local network.
- The operator can open the Lake web user interface on one computer.

Postconditions (Success Guarantees):

- A cluster exists containing the operator's computer (the leader node) and every selected computer whose owner confirmed.
- The cluster view shows each member's memory and platform.

Postconditions (Failure Guarantees):

- No computer is in the cluster without its owner's confirmation.
- Computers that did not join are unaffected and keep running as before.
- The leader node's interface displays the error to the user.

Main Success Scenario (Basic Flow):

1. The operator starts the Lake client on each computer they want to use.
2. On one of them, the operator opens the Web UI, and that computer becomes the leader node.
3. Lake lists the other computers on the network that are running the Lake client.
4. The operator selects the computers to include.
5. Lake asks each selected computer's owner to confirm (UC2).
6. Lake shows the cluster with each computer's memory and platform.

Extensions:

- **3a.** No other computers running the Lake client are found: Lake tells the operator none were found. The operator can start the Lake client on other computers and refresh the list.

- **5a.** An owner declines: that computer is left out of the cluster and the operator is told which ones declined (UC2).
- **5b.** An owner does not respond: the request ends after a timeout, the computer is left out, and the operator is notified.
- **5c.** A selected computer becomes unreachable before it confirms: it is left out and the operator is notified.
- **6a.** No computer confirmed: the cluster contains only the leader node and the operator is notified.

Justification: This is the first step of any session with the system and is required to happen before any other actions can take place. It also validates that Lake will have consent confirmation from each of the computers in the network in advance.

UC2: Confirm a device joining

Use Case Name: Confirm a device joining
Primary Actor: Device Owner
Stakeholders and Interests:

- Device Owner: decides whether their computer's memory, compute, and bandwidth are used, and wants to see what it is doing afterwards.
- Operator: needs to know which computers actually joined.

Preconditions:

- The Lake client is running on the owner's computer.
- An operator has selected the computer for a cluster, either when forming one (UC1) or during a session (UC6).

Postconditions (Success Guarantees):

- If accepted, the device is a member of the named cluster and its owner can see the device's performance and what it is computing.
- If declined, the device is not a member.

Postconditions (Failure Guarantees):

- The device does not take part in the cluster without an explicit acceptance.

- The operator is told the outcome for this device.

Main Success Scenario (Basic Flow):

1. A prompt appears on the device asking whether it may join the named cluster.
2. The owner accepts.
3. The device joins the cluster.
4. The owner can see what the device is doing for the cluster.

Extensions:

- **2a.** The owner declines: the device is left out of the cluster and the operator is notified.
- **2b.** The owner does not respond: the request expires, the device is left out, and the operator is notified.
- **2c.** The owner is not at the device or the device is asleep: the device is treated as unavailable and left out.

Justification: Consent is a stated requirement of the first goal (FR.G.4.1), and the owner is a direct stakeholder who may not be the person running the cluster.

UC3: Run a model across the cluster

Use Case Name: Run a model across the cluster
Primary Actors: Operator (to load the model); Inference User (to send prompts)
Stakeholders and Interests:

- Operator: wants a model larger than any one computer can hold to load across the cluster.
- Inference User: wants a complete, correct response to each prompt, streamed as it is generated.
- Device Owner: wants their device to hold only its share of the model.

Preconditions:

- A cluster exists (UC1).

- The operator has a model file in a supported format (GGUF).
- The cluster has enough combined memory for the model.

Postconditions (Success Guarantees):

- The model is loaded across the cluster, each node holding only its assigned layers.
- The user receives a complete response to each prompt, and the session keeps its context for further prompts.

Postconditions (Failure Guarantees):

- If the model cannot be loaded, the operator is notified including a reason why.
- A partial response from the device network is identified as such with a warning.

Main Success Scenario (Basic Flow):

1. The operator chooses a model file.
2. Lake divides the model among the nodes and each node loads its share.
3. Lake reports that the model is loaded across the cluster.
4. A user sends a prompt.
5. The response streams back to the leader node.
6. Steps 4 and 5 repeat until the user chooses to stop the session.

Extensions:

- **1a.** The chosen file is not a supported model format or is unreadable: Lake rejects it and provides reasoning.
- **2a.** The cluster's combined memory is too small for the model: Lake does not start loading, tells the operator why, and says what would fix it (for example adding a computer or choosing a smaller model).
- **2b.** A node fails while loading its share: Lake treats it as a node leaving (UC5).
- **4a.** A prompt arrives before loading has finished: Lake notifies the user the model is not ready.
- **5a.** A node leaves while the response is streaming: see UC5.

Justification: This is the primary goal of Lake: running a model that does not fit on any single device.

UC4: Use Lake from an existing tool

Use Case Name: Use Lake from an existing tool
Primary Actor: Inference User
Stakeholders and Interests:

- Inference User: wants to keep using the chat interface or coding assistant they already use, with no changes to it.
- Regulated organisations: want prompts and responses to stay only on the local network.

Preconditions:

- A cluster has a model loaded (UC3).
- The user's tool can talk to an OpenAI-format API and can reach the leader node over the network.
- The user knows the leader node's API address.

Postconditions (Success Guarantees):

- The tool works in a similar fashion to how it would with a cloud provider.
- Prompts and responses pass only through devices on the local network.

Postconditions (Failure Guarantees):

- If the request cannot be served, the tool receives an error response in the API's format rather than a hung connection.
- No request data is sent outside the cluster.

Main Success Scenario (Basic Flow):

1. The user opens the chat interface or coding assistant they already use.
2. The user points it at the Lake address instead of a cloud provider.
3. The user uses the tool as before.
4. Lake receives each prompt, runs it across the cluster, and returns the response (streamed if the tool asks for streaming).

Extensions:

- **2a.** The address is wrong or the leader node is unreachable: the tool reports a connection error.
- **3a.** No model is loaded: Lake returns an error saying so instead of waiting.
- **3b.** The tool uses tool calling or another option Lake does not support: Lake returns an error naming the unsupported option.
- **4a.** A node leaves during the request: see UC5.

Justification: Compatibility with the tools people already use removes the main adoption barrier and is the main route by which inference users reach Lake.

UC5: Keep working when a device leaves

Use Case Name: Keep working when a device leaves
Primary Actor: Operator (informed by Lake); Inference User (whose request is affected)
Stakeholders and Interests:

- Inference User: wants the current prompt answered without having to resubmit it, and never wants a cut-off answer presented as a complete one.
- Operator: wants to know which device left and what actions will be taken.
- Device Owner: wants to be able to close or disconnect their device at any time.

Preconditions:

- A cluster with at least two nodes has a model loaded and is serving a session.

Postconditions (Success Guarantees):

- The model is reassigned across the remaining nodes.
- The user's current prompt has been answered, or re-run, without being resubmitted.

Postconditions (Failure Guarantees):

- If the remaining nodes cannot hold the model, the operator is told why and what would fix it.

Main Success Scenario (Basic Flow):

1. During a session, a computer in the cluster is closed or disconnected by its owner.
2. Lake detects that the node has left.
3. Lake notifies the operator which device left and that it is reorganizing.
4. Lake reassigns the model's layers across the remaining nodes.
5. Lake answers or re-runs the user's current prompt without the user resubmitting it.
6. The session continues on the remaining nodes.

Extensions:

- **2a.** The device fails or loses its connection without warning: Lake detects it as in step 2 and continues.
- **4a.** The remaining computers cannot hold the model: Lake stops serving, notifies the operator, and provides a potential solution (for example adding a computer), and does not leave the session hanging.
- **4b.** The leader node itself leaves: the session cannot continue, and the operator must start again from UC1.

Justification: Resilience to devices leaving is a key differentiator between Lake and existing local distributed inference providers.

UC6: Add a device during a session

Use Case Name: Add a device during a session
Primary Actor: Operator
Stakeholders and Interests:

- Operator: wants to use a newly available computer without restarting the session.
- Device Owner: wants to consent before the device joins (UC2).
- Inference User: wants to continue prompting and benefit from the added computer.

Preconditions:

- A cluster is running a session.
- A new computer running the Lake client has appeared on the network.

Postconditions (Success Guarantees):

- The new computer is a member of the cluster and holds part of the model.
- The operator sees the new arrangement.

Postconditions (Failure Guarantees):

- If the computer does not join, the cluster and the session continue unchanged.
- The user's requests are not lost or corrupted by the attempt.

Main Success Scenario (Basic Flow):

1. The operator selects a new computer that has appeared on the network.
2. Its owner confirms (UC2).
3. Lake reorganises the model's layers to use the new computer.
4. Lake shows the new device network arrangement.
5. The user continues with the session including the new device.

Extensions:

- **2a.** The owner declines or does not respond: the computer is not added and the session continues unchanged.
- **3a.** Reorganisation fails: Lake keeps the previous arrangement, tells the operator, and the session continues.
- **3b.** A prompt is in progress during reorganisation: Lake finishes it or re-runs it without the user resubmitting, and never presents a partial answer as complete.
- **5a.** The change in speed is not an improvement: the operator can inspect the cluster view (UC7).

Justification: Taking advantage of new devices is a key differentiator between Lake and existing local distributed inference providers. It allows Lake to improve performance through dynamic allocation as a session progresses.

UC7: Find the bottleneck

Use Case Name: Find the bottleneck
Primary Actor: Operator
Stakeholders and Interests:

- Operator: wants to know why responses are slow and what to change.
- Inference User: wants faster responses.

Preconditions:

- A cluster has a model loaded and has served at least one request.

Postconditions (Success Guarantees):

- The operator knows which computer or connection is the slowest and can decide what to change.

Postconditions (Failure Guarantees):

- Viewing measurements does not interrupt or alter the running session.

Main Success Scenario (Basic Flow):

1. Responses feel slow, so the operator opens the cluster view.
2. The view shows how long each computer and each connection takes per word generated.
3. The view marks the slowest computer or connection.
4. The operator takes action depending on the provided information, ideally improving the performance of the network.

Extensions:

- **2a.** A node is not reporting measurements: the view marks it as unavailable.
- **4a.** The operator removes or adds a computer: see UC5 or UC6.

Justification: Splitting a model over a network can be slower than one device only, so the operator must see whether pooling is beneficial and potential areas for improvement.

UC8: Serve several users at once

Use Case Name: Serve several users at once
Primary Actor: Inference User (two or more)
Stakeholders and Interests:

- Inference Users: each wants their own response, as fast as if they were alone, and wants no other user to see their prompt or answer.
- Operator: wants to see when the cluster is busy enough that requests queue.
- Regulated organisations: want requests kept separate for data privacy.

Preconditions:

- A cluster has a model loaded (UC3).
- Each user's tool can reach the leader node (UC4).

Postconditions (Success Guarantees):

- Every user receives a complete response to their own prompt.
- No user's prompt or response is visible to another user.

Postconditions (Failure Guarantees):

- A request that cannot be served is queued or returns an error.
- One user's request is never visible or lost to another user, requests are always isolated between users.

Main Success Scenario (Basic Flow):

1. Two or more users send prompts to the same cluster at the same time, from their own tools.
2. Lake processes the requests concurrently, each at a different stage of the model, keeping their state separate.
3. Each user sees their own response stream back.
4. Each response arrives roughly as fast as it would have alone.

Extensions:

- **2a.** The cluster is busy enough that further requests cannot start immediately: Lake queues them, and the operator sees the queue in the cluster view.
- **3a.** A node leaves while several requests are in progress: see UC5; each affected request is re-run or reported individually.
- **4a.** Responses slow because of load: the operator can inspect the cluster view (UC7).

Justification: Serving several users at once is one of the main advantages of splitting a model by layers, and it is where Lake's throughput gain is clear.

S.7 Prioritization

Priorities follow MoSCoW: **M** must have, **S** should have, **C** could have, **W** won't have (this release). They are assigned with the following rules:

- **M**: needed to meet one of the goals in Section G.3.
- **S**: improves a core goal, but the system still meets that goal without it.
- **C**: belongs to a stretch goal in the Problem Statement, or is an optimisation with no effect on the core goals.
- **W**: explicitly deferred to a later release. No requirement is currently W.

All priorities in Tables 8–17 were last decided on 2026-10-16.

ID	Requirement	Priority	Reasoning
FR.S.1.1	Provide local partition	M	Each node must load its own partition for a split model to run at all.
FR.S.1.2	Collect device metrics	S	Feeds failure detection and the dashboard, but inference runs without it.
FR.S.1.3	Collect network metrics	S	Same as FR.S.1.2, for connections between nodes.
FR.S.1.4	Discover devices	M	A cluster cannot be formed without finding the other nodes.
FR.S.1.5	Neighbour info to Tensor I/O	M	Each node must know where to send its output tensors.

Table 8: Prioritisation of Coordinator requirements on all nodes

ID	Requirement	Priority	Reasoning
FR.S.1.6	Send partitions	M	Distributing partitions is the core of model parallelisation.
FR.S.1.7	Monitor cluster	S	Needed for the dashboard and failure detection, not for inference.
FR.S.1.8	Inputs to Model Partitioner	M	The Model Partitioner needs the available nodes and their memory to produce a valid scheme.
FR.S.1.9	OpenAI-compatible API	M	The only way users submit prompts.
FR.S.1.10	Detect failing followers	M	Dynamic allocation cannot react to failures it does not detect.
FR.S.1.11	Rebuild on failure or leave	M	Core of the Dynamic Allocation goal.
FR.S.1.12	Rebuild on join	S	A join only adds capacity; the cluster keeps working without it.
FR.S.1.13	Retry after rebuild	S	Improves resilience, but the user can resubmit a failed request.
FR.S.1.14	Concurrent requests	M	Required by the concurrent inference goal. This means serving requests concurrently, not only accepting and queueing them.
FR.S.1.15	Send cluster graph	M	Followers must know the pipeline order to connect to each other.
FR.S.1.16	Web UI data and control	M	The Web UI is how users create and inspect clusters.
FR.S.1.17	Wait for confirmations	M	Required to keep untrusted devices out of a cluster (NFR.S.12).
FR.S.1.18	Selected followers only	M	Same as FR.S.1.17.

Table 9: Prioritisation of Coordinator requirements on the leader node

ID	Requirement	Priority	Reasoning
FR.S.1.19	Accept assignments	M	A follower must accept its partition to take part.
FR.S.1.20	Ask user to confirm	M	Required to keep untrusted devices out of a cluster (NFR.S.12).
FR.S.1.21	Confirmation required	M	Same as FR.S.1.20.
FR.S.1.22	No reconfirm on rebuild	S	Avoids interrupting a rebuild, but the user could confirm again by hand.
FR.S.1.23	Report metrics	S	Same as FR.S.1.2.
FR.S.1.24	Detect leader failure	S	Without it, followers keep holding resources after the leader fails, but no data is lost.
FR.S.1.25	Shut down on leader failure	S	Same as FR.S.1.24.

Table 10: Prioritisation of Coordinator requirements on follower nodes

ID	Requirement	Priority	Reasoning
FR.S.2.1	Fit in memory	M	A partition that does not fit crashes the node.
FR.S.2.2	Report infeasible split	M	The user must know when the cluster cannot hold the model.
FR.S.2.3	Equivalent to original	M	A split that changes the model produces wrong output.
FR.S.2.4	Size by capability	S	Improves speed; a split based on memory alone still works.

Table 11: Prioritisation of Model Partitioner requirements

ID	Requirement	Priority	Reasoning
FR.S.3.1	Load partition	M	Each node must hold its layers in GPU-accessible memory to run them.
FR.S.3.2	Run incoming tensors	M	Core of model parallelisation.
FR.S.3.3	Send output tensors	M	Core of model parallelisation.
FR.S.3.4	Measure performance	S	Same as FR.S.1.2.
FR.S.3.5	Concurrent requests	M	Same as FR.S.1.14; each node must process tensors from several requests.

Table 12: Prioritisation of Inference Engine requirements

ID	Requirement	Priority	Reasoning
FR.S.4.1	Connect to neighbours	M	Tensors cannot pass between nodes without a connection.
FR.S.4.2	Adaptive compression	C	An optimisation; tensors can be sent uncompressed.
FR.S.4.3	Receive tensors	M	Core of model parallelisation. Decompression applies only if FR.S.4.2 is implemented.
FR.S.4.4	Send tensors	M	Same as FR.S.4.3.
FR.S.4.5	Measure network	S	Same as FR.S.1.3.
FR.S.4.6	Request association	M	Without it, tensors from concurrent requests mix and outputs interfere.

Table 13: Prioritisation of Tensor I/O requirements

ID	Requirement	Priority	Reasoning
FR.S.5.1	Scan for devices	M	Users must see available devices to build a cluster.
FR.S.5.2	Select devices	M	Users choose which devices to trust (NFR.S.12).
FR.S.5.3	Select model file	M	There is nothing to run without a model.
FR.S.5.4	Infeasible split error	M	The user must know why a cluster cannot start.
FR.S.5.5	Display metrics	S	Supports understandability (NFR.S.1), not inference.
FR.S.5.6	Highlight bottlenecks	C	Useful, but users can read bottlenecks from the metrics in FR.S.5.5.
FR.S.5.7	Invite prompt	M	Same as FR.S.1.20.

Table 14: Prioritisation of Web UI requirements

ID	Requirement	Priority	Reasoning
NFR.S.1	Understandability	S	Matters for adoption, but the system works without it.
NFR.S.2	Controllability	S	Same as NFR.S.1.
NFR.S.3	Installability	S	Users can build from source until installers exist.
NFR.S.4	Model size	M	The primary metric of the Improved Local Performance goal.
NFR.S.5	Scalability	M	The measure of the concurrent inference goal (NFR.G.4.6).
NFR.S.6	Intelligence	M	A split model that answers worse defeats the purpose of Lake.
NFR.S.7	Consistency	C	Cosmetic.
NFR.S.8	Reject bad model files	M	Prevents crashes and misleading output from a corrupted file.
NFR.S.9	Packet loss	S	The transport protocol already handles most loss and corruption.
NFR.S.10	No stored request data	M	Privacy is a primary reason to use Lake.
NFR.S.11	No data leaves the cluster	M	Same as NFR.S.10. The developer we interviewed would not use Lake with telemetry.
NFR.S.12	No untrusted devices	M	Same as NFR.S.10.
NFR.S.13	Operating systems	M	Required by the Cross-Platform Optimisation goal.
NFR.S.14	CPU architectures	M	Same as NFR.S.13; covers Apple Silicon and x86 machines.
NFR.S.15	GPU vendors	M	Same as NFR.S.13; the Problem Statement supports NVIDIA, AMD and Apple GPUs.

Table 15: Prioritisation of non-functional requirements

ID	Requirement	Priority	Reasoning
NFR.E.1	Supported platforms	M	Required by the Cross-Platform Optimisation goal. The Android and iOS part is a stretch goal and is C.
NFR.E.2	Model layer partitioning	M	The core of model parallelisation.
NFR.E.3	OpenAI API specification	M	Same as FR.S.1.9.
NFR.E.4	Model weights unchanged	M	Modified weights produce wrong output.
NFR.E.5	No third-party services	M	Privacy is a primary reason to use Lake.
NFR.E.6	Formatting and linting	S	Keeps the code maintainable, but does not change behaviour.
NFR.E.7	GPL compliance	M	A licence violation would prevent Lake from being distributed.
NFR.E.8	Model licences	M	Same as NFR.E.7, for the models Lake runs.

Table 16: Prioritisation of environmental requirements

ID	Requirement	Priority	Reasoning
SR0-1	Discard stale health data	S	Supports failure detection, which already works from timeouts (SR4-1).
SR0-2	Route only to assigned nodes	M	Sending tensors to a node without the right partition produces wrong output.
SR1-1	At least two nodes	M	Distributed inference on a single node is not distributed. A model that runs on one device is used normally instead.
SR1-2	All layers assigned	M	A missing layer produces wrong output.
SR1-3	Memory safety margin	M	Same as FR.S.2.1.
SR2-1	Bounded recovery	S	Same as FR.S.1.13. Reporting the failure keeps a request from waiting forever.
SR3-1	Detect corrupted tensors	M	A corrupted tensor produces wrong output.
SR3-2	Report decoding failures	S	Helps recovery and debugging; SR3-1 already prevents wrong output.
SR4-1	Communication timeouts	M	Without timeouts, a lost connection stalls the cluster indefinitely.
SR4-2	Mark unhealthy nodes	M	Same as FR.S.1.10.
SR4-3	Latency baseline	C	Only reports slowness; inference is unaffected.
SR5-1	Two-sided approval	M	Same as NFR.S.12.
SR5-2	No sensitive data in logs	M	Same as NFR.S.10.

Table 17: Prioritisation of safety and security requirements

S.8 Verification and Acceptance Criteria

Named values such as *HEALTH_INTERVAL* are defined in the Symbolic Constants appendix. The detailed test cases for each group are given in the V&V Plan.

S.8.1 Validation

Validation checks that Lake is the right system for its stakeholders, measured against the goals in the Problem Statement. The team demonstrates the system at the proof of concept, Revision 0 and final demonstrations, and the teaching assistant and other reviewers judge it against these goals:

- **Model Parallelisation**: a model split across two or more nodes produces coherent output (groups 2 and 3 below).
- **Improved Local Performance**: the cluster runs a model that no single node can hold (group 3).
- **Cross-Platform Optimisation**: a cluster mixing operating systems runs a model (group 12).
- **Dynamic Allocation**: the cluster keeps serving requests after nodes join and leave, and the reallocation time is recorded (group 6).
- **Concurrent Inference**: the cluster serves at least *MIN_CONCURRENT* users at once without their outputs interfering (group 13).

A few people who are not on the team also try the system in the usability session (group 10), to confirm that it is understandable to new users.

S.8.2 Verification

Each group below lists the requirements it covers, how they are verified, and what counts as acceptance. Every requirement in S.2, the Environmental Requirements and the Safety and Security Requirements belongs to at least one group.

1. **Partitioning.** FR.S.1.8, FR.S.2.1, FR.S.2.2, FR.S.2.4, FR.S.5.4, NFR.E.2, NFR.E.4, SR1-1–SR1-3.
   Method: unit and property-based tests of the Model Partitioner on generated clusters and models; a Web UI test with a model too large for the selected devices; a checksum of the model file before and after partitioning.

Acceptance: every scheme produced is valid by Definition S.9.2.2, leaves at least *MEMORY_MARGIN* of each node's memory free, and uses at least two nodes for distributed inference; failure is reported exactly when no such scheme exists, and the Web UI shows the error; the model file's checksum is unchanged. For FR.S.2.4, until its definition of capability is made measurable: of two nodes with equal memory, the one with the higher measured throughput never receives fewer layers.

2. **Inference correctness.** FR.S.1.1, FR.S.1.5, FR.S.1.6, FR.S.1.15, FR.S.1.19, FR.S.2.3, FR.S.3.1–FR.S.3.3, FR.S.4.1, FR.S.4.3, FR.S.4.4, NFR.S.6.
   Method: an integration test in continuous integration that splits a small model across two processes and compares the output with an unsplit run of the same model; benchmark and perplexity runs on team hardware.
   Acceptance: on a single platform, the split model produces the same tokens as the unsplit model under greedy decoding. Across platforms, and on every benchmark, the split model scores at least *BENCH_RATIO* of the unsplit model's score.

3. **Model capacity.** NFR.S.4.
   Method: a system test on team hardware with a model whose memory need exceeds the free memory of the largest node.
   Acceptance: the cluster runs the model and meets the acceptance criteria of group 2.

4. **Cluster formation and trust.** FR.S.1.4, FR.S.1.17, FR.S.1.18, FR.S.1.20, FR.S.1.21, FR.S.5.1–FR.S.5.3, FR.S.5.7, NFR.S.12, SR5-1.
   Method: an integration test on a local network with at least three devices running Lake: one selected device that confirms, one that declines, and one that is not selected. Traffic is captured on the network.
   Acceptance: all three devices are discovered and listed; inference does not start until every selected device has confirmed; the declining and unselected devices never receive a partition or a tensor, so a device joins only with approval on both the leader and the device itself.

5. **Monitoring.** FR.S.1.2, FR.S.1.3, FR.S.1.7, FR.S.1.16, FR.S.1.23, FR.S.3.4, FR.S.4.5, FR.S.5.5, SR0-1, SR4-3.
   Method: an integration test that records when each metric report is sent and received, including delayed reports and added network latency.
   Acceptance: consecutive reports arrive every *HEALTH_INTERVAL*, within *INTERVAL_TOL*; reports older than *HEALTH_TTL* are not used to judge

availability; latency more than *LATENCY_TOL* above a connection's baseline is reported to the Coordinator; and the Web UI shows the reported values for the cluster, each node and each connection.

6. **Dynamic allocation.** FR.S.1.10–FR.S.1.13, FR.S.1.22, FR.S.1.24, FR.S.1.25, SR0-2, SR2-1, SR4-1, SR4-2.
   Method: fault-injection system tests that stop a follower during a request, throttle a follower below *FAIL_THRESHOLD* of its usual throughput, add a node while idle and while generating, and stop the leader.
   Acceptance: the system moves between states as in the allocation state machine (S.9.3); a retried request completes and meets the criteria of group 2, or fails with an error returned through the API after at most *RECOVERY_LIMIT* attempts; a connection silent for *NODE_TIMEOUT* is treated as failed; a removed node stays unavailable until it passes a health check; tensors are only sent to nodes in the current partitioning scheme; a rejoining node is not asked to confirm again; followers shut down within *LEADER_TIMEOUT* plus *HEALTH_INTERVAL* of the leader stopping. The reallocation time is recorded for the Performance Report.

7. **Fault tolerance.** NFR.S.8, NFR.S.9, SR3-1, SR3-2.
   Method: loading truncated, corrupted and non-GGUF files; running inference with network emulation that corrupts packets and drops up to *MAX_LOSS* of them; corrupting and truncating tensors between nodes on purpose.
   Acceptance: bad files are rejected with a message before any partition is sent; output under packet loss is identical to output without it; every corrupted or truncated tensor is detected and discarded, the failure is logged and reported to the Coordinator, and the request is retried.

8. **Privacy.** NFR.S.10, NFR.S.11, NFR.E.5, SR5-2.
   Method: after a set of requests, every file Lake wrote, including diagnostic logs, is searched for the prompts and responses; all traffic from every node is captured during requests, while idle and during any update check.
   Acceptance: no request data remains on disk after its request completes. Logs contain no prompts, responses, tensor data or KV cache content. Every captured packet goes to a cluster node, a local device being discovered or invited, or the client that sent the request.

9. **API.** FR.S.1.9, NFR.E.3.
   Method: requests through the official OpenAI client library and through a third-party tool such as OpenCode.

Acceptance: chat completion requests, both streamed and not streamed, succeed and their responses are parsed without errors.

10. **Usability.** NFR.S.1–NFR.S.3, FR.S.5.6.
    Method: for NFR.S.1 and NFR.S.2, a short task-based session with at least *TEST_USERS* people who have not used Lake, such as students from other capstone teams. The team verifies the rest itself: an install test on each operating system for NFR.S.3, and a throttled node for FR.S.5.6.
    Acceptance: within *UI_TASK_TIME* of first seeing the interface, each participant can create a cluster and can name the participating devices, the token throughput and the bottleneck; the throttled node is highlighted; installation takes at most *INSTALL_CLICKS* clicks on Windows and macOS, and one package manager command on Linux.

11. **Look and feel.** NFR.S.7.
    Method: inspection of the Web UI source.
    Acceptance: until the requirement is made measurable, every colour and font used comes from the Web UI's theme definition.

12. **Compatibility.** NFR.S.13–NFR.S.15, NFR.E.1.
    Method: continuous integration builds and unit tests on Windows, macOS and Linux runners; system tests on team devices covering x86_64 and ARM processors and NVIDIA, AMD and Apple GPUs.
    Acceptance: every build and unit test passes on each runner, and the group 2 criteria hold on each processor and GPU combination tested.

13. **Concurrent requests.** FR.S.1.14, FR.S.3.5, FR.S.4.6, NFR.S.5.
    Method: load tests with increasing numbers of concurrent requests on clusters of increasing size.
    Acceptance: every cluster serves at least *MIN_CONCURRENT* requests at the same time; no tokens from one request appear in another; and the number of concurrent requests served with at most *MAX_DEGRADATION* loss of per-request throughput grows in proportion to the number of nodes.

14. **Tensor compression.** FR.S.4.2.
    Method: throughput measurements with adaptive compression on and off, under several network speeds.
    Acceptance: until the requirement is made measurable, throughput with adaptive compression is never lower than with compression off.

15. **Code quality and licensing.** NFR.E.6–NFR.E.8.
Method: the linting and formatting stage of continuous integration; an automated audit of dependency licences; a review of the licence of each model used in testing and demonstrations.
Acceptance: every merged pull request passes the linting and formatting stage; no dependency has a licence incompatible with GPL v3.0; every model used is permitted for that use by its licence.

S.9 Formalisations

Two parts of Lake are formalised. S.9.2 defines what counts as a valid partitioning scheme for the model across the cluster, which every other requirement depends on. S.9.3 specifies, as a finite state machine, how the system responds when nodes join and leave the cluster. This is the precise form of the Dynamic Allocation goal.

S.9.1 Notation

For integers a and b, $a..b = \{\, n \in \mathbb{N} \mid a \leqslant n \leqslant b \,\}$, which is empty when $a > b$.

Table 18: Symbols used in the formalisation

Symbol	Type (unit)	Meaning
N	$\mathbb{N}$, $N \geqslant 1$	Number of partition units in the model (embedding, transformer layers and output head), in execution order.
L	$L = 1..N$	The set of partition units.
D	finite set	The nodes currently in the cluster: devices the user has selected (FR.S.5.2) and that have confirmed participation (FR.S.1.21).
m_ℓ	$\mathbb{N}$ (bytes)	Memory needed to hold unit ℓ's weights and its KV cache for $CONTEXT_MAX$ tokens.
M_d	$\mathbb{N}$ (bytes)	GPU-accessible memory free on node d when it joins the cluster, minus the runtime overhead reserved by FR.S.2.1.

The formalisation assumes that M_d does not change while d is in the cluster, and that $join(d)$ only occurs for $d \notin D$ and $leave(d)$ only for $d \in D$.

S.9.2 Valid Partitioning Schemes

Definition S.9.2.1 (Partitioning scheme)**.** A *partitioning scheme* is a finite sequence of partitions

$$P = \langle (d_1, a_1, b_1), \ldots, (d_k, a_k, b_k) \rangle, \qquad k \geqslant 1,$$

where d_i is a node and $a_i, b_i \in \mathbb{N}$. Partition i consists of units $a_i..b_i$, runs on node d_i, and passes its output to partition $i+1$. The nodes of P are $dev(P) = \{d_1, \ldots, d_k\}$.

Definition S.9.2.2 (Valid partitioning scheme)**.** P is *valid* over cluster D, written $P \in \mathcal{V}(D)$, if and only if:

(V1) $dev(P) \subseteq D$ and $d_i \neq d_j$ for all $1 \leqslant i < j \leqslant k$;

(V2) $a_1 = 1$ and $b_k = N$;

(V3) $a_i \leqslant b_i$ for all $i \in 1..k$;

(V4) $a_{i+1} = b_i + 1$ for all $i \in 1..k-1$;

(V5) $\sum_{\ell=a_i}^{b_i} m_\ell \leqslant M_{d_i}$ for all $i \in 1..k$.

(V1) gives each node at most one partition. (V2)–(V4) make the partitions consecutive, non-empty blocks of units in execution order. (V5) says each node has room for its partition. The Model Partitioner must return a member of $\mathcal{V}(D)$ whenever $\mathcal{V}(D) \neq \emptyset$, and must report failure otherwise (FR.S.2.2).

Lemma S.9.2.3 (Exact cover)**.** *If $P \in \mathcal{V}(D)$, every unit $\ell \in L$ belongs to exactly one partition. That is, the sets $a_i..b_i$ for $i \in 1..k$ are non-empty, pairwise disjoint, and their union is L.*

Proof. Let $b_0 = 0$. By (V2) and (V4), $a_i = b_{i-1} + 1$ for all $i \in 1..k$, so partition i consists of units $(b_{i-1}+1)..b_i$. By (V3) these sets are non-empty, so $0 = b_0 < b_1 < \cdots < b_k = N$. Each $\ell \in L$ therefore satisfies $b_{i-1} < \ell \leqslant b_i$ for exactly one i, so it belongs to exactly one partition. □

S.9.3 Allocation State Machine

The system's response to nodes joining and leaving the cluster is a finite state machine (Q, Σ, δ, q_0) whose transitions are guarded by predicates over the cluster D and the active partitioning scheme P.

S.9.3.1 States. $Q = \{\mathsf{U}, \mathsf{A}, \mathsf{I}, \mathsf{G}, \mathsf{R}, \mathsf{X}\}$ with $q_0 = \mathsf{U}$:

- U (Unloaded): no model is loaded.
- A (Allocating): a target scheme P' is being loaded, and no request is in progress.
- I (Idle): the model is loaded on scheme P.
- G (Generating): a request is being served on P.
- R (Recovering): a node in $dev(P)$ left during a request. The request is stopped while P' is loaded, then retried (FR.S.1.13).
- X (Insufficient): $\mathcal{V}(D) = \emptyset$, so the cluster cannot hold the model.

S.9.3.2 Events. $\Sigma = \{load, unload, req, fin, done\} \cup \{join(d), leave(d)\}$:

- *load* and *unload*: the user creates a cluster with a model, or shuts it down.
- *req* and fin: a request is submitted, or finishes.
- *done*: the system finishes loading P'.
- $join(d)$: a device the user selected confirms participation and is added to the cluster.
- $leave(d)$: follower d leaves, or is detected as failing (FR.S.1.10).

This finite state machine only models the behaviour of follower nodes. If the leader node is detected as failed (FR.S.1.24), the followers shut down and the session ends (FR.S.1.25), which has the same effect as *unload*.

S.9.3.3 Guards. Let D' be the cluster after the event: $D \cup \{d\}$ for $join(d)$, $D \setminus \{d\}$ for $leave(d)$, and D otherwise. Then

$$\varphi \equiv \mathcal{V}(D') \neq \emptyset \qquad \text{and} \qquad \psi \equiv d \in dev(P).$$

Whenever the machine enters A or R, it selects a new target $P' \in \mathcal{V}(D')$.

S.9.3.4 Transitions. Table 19 defines δ. Combinations it does not list are handled as follows:

- *unload* moves every state to U and aborts any request in progress.
- *leave*(d) with $\neg\psi$ in I or G removes d from D and leaves the state unchanged.
- *req* is rejected with an error in U and X. *req* in A, G or R, and *join*(d) in G or R, are queued and fire in arrival order once I is entered. The machine therefore tracks one request at a time, which is a simplification of concurrent inference (FR.S.1.14). With several requests, G means at least one request is in progress on P. *req* in G then starts another request instead of being queued, fin returns to I only when the last request finishes, and *leave*(d) with ψ in G or R stops and retries every request, since they all run on P.
- Any other combination leaves the state unchanged.

Table 19: Transition function δ of the allocation state machine

#	State	Event [guard]	Next	Effect
1	U	*load* [φ]	A	Start loading P'.
2	U	*load* [$\neg\varphi$]	X	Report a partitioning failure.
3	A	*done*	I	$P := P'$.
4	A	*join*(d), *leave*(d) [φ]	A	Reselect P' and restart loading.
5	A	*join*(d), *leave*(d) [$\neg\varphi$]	X	Report a partitioning failure.
6	I	*req*	G	Start generating.
7	I	*join*(d)	A	Rebuild the cluster over D'.
8	I	*leave*(d) [$\psi \wedge \varphi$]	A	Rebuild the cluster over D'.
9	I	*leave*(d) [$\psi \wedge \neg\varphi$]	X	Report a partitioning failure.
10	G	fin	I	Return the response.
11	G	*leave*(d) [$\psi \wedge \varphi$]	R	Stop the request.
12	G	*leave*(d) [$\psi \wedge \neg\varphi$]	X	Abort the request with an error.
13	R	*done*	G	$P := P'$, retry the request.
14	R	*leave*(d) [φ]	R	Reselect P' and restart loading.
15	R	*leave*(d) [$\neg\varphi$]	X	Abort the request with an error.
16	X	*join*(d) [φ]	A	Start loading P'.

Row 7 needs no guard. Conditions (V1)–(V5) refer to D only through $dev(P) \subseteq D$, so a scheme valid over D stays valid over $D \cup \{d\}$. For the same reason, removing a node outside $dev(P)$ keeps P valid.

S.9.3.5 Invariant. In states I and G, $P \in \mathcal{V}(D)$. P is only assigned in rows 3 and 13, from a P' that is reselected after every change to the cluster. The only change that keeps the machine in I or G removes a node outside $dev(P)$.

S.10 Likely Changes to Requirements

- **LC.1** A Lake cluster cannot recover from a failing leader node (see FR.S.1.25). In the future, the ability to recover from a failing leader node may be added.
- **LC.2** Lake only supports desktop operating systems (Windows, macOS, Linux) (see NFR.S.13). Support for mobile devices (iOS, Android) may be added once other goals and requirements have been met.
- **LC.3** Lake assumes the local area network being used to connect the devices is private and trusted (See Hazard Analysis Section 4 - Critical Assumptions). Lake's may be extended with additional security hardening to allow clusters to run on public or untrusted networks.
- **LC.4** Lake requires that each device manually confirm joining a cluster (see FR.S.1.17 and FR.S.1.21). In the future, a trust mechanism may be introduced, which would allow devices to automatically join clusters created by trusted devices without requiring confirmation.
- **LC.5** Requirement thresholds and intervals will likely change, as real-world usage gives better estimates for reasonable threshold values.
- **LC.6** It is likely that future versions of Lake will support a headless mode where all UI interactions are performed through a command line interface. This would be useful for users wishing to use headless servers as nodes.

S.11 Unlikely Changes to Requirements

- **UC.1** Model training and fine-tuning support is unlikely to be added to Lake. Lake is primarily an inference framework, so the addition of training would be out of scope for such a project.
- **UC.2** Additional input model formats aside from GGUF are unlikely to be added, as GGUF is one of the most common standards for LLMs. Additionally, models in other formats can be converted to GGUF so there is little reason for Lake to support them directly.

- **UC.3** A primary goal of Lake is privacy (see NFR.S.11), so modifications that would compromise privacy like incorporating cloud-based services are unlikely.
- **UC.4** Even though one of Lake's goals is to run larger models, integrating quantisation tools into Lake is unlikely. Lake's scope is focused on distributed inference, and quantisation tools would better fit in the scope of an independent project.

(P) Project

P.1 Roles and Personnel

- **Liaison (Ethan Walsh)** – Coordinates communication between the team and other project stakeholders and faculty.
- **Repository administrator (Matthew Nesbitt)** – Manages repository and organizational configurations.
- **Architecture lead (Ethan Walsh)** – Leads the architecture design of the system.
- **Coordinator lead (Gary Qin)** – Leads the coordinator and inter-process communication (IPC) design of the system.
- **Performance lead (Matthew Nesbitt)** – Leads the performance benchmarking and optimization of the system.
- **Machine learning leads (Jinwoo Hong, Colin Chambachan)** – Lead the ML systems of Lake.
- **Meeting chair (Rotating weekly amongst team members)** – Leads weekly team meetings.
- **QA lead (Rotating weekly amongst team members)** – Performs quality assurance and dogfooding of the system.
- **Teaching assistant (Rebecca Di Filippo)** – Provides technical guidance and feedback throughout the project.

P.2 Imposed Technical Choices

- **API specification** – Lake will follow the OpenAI API specification for its request and response formats (**?**). This is chosen due to the widespread adoption of this API specification and the flexibility it provides when connecting to existing external clients.
- **Supported platforms** – Windows, macOS, and Linux will be supported platforms for nodes within the Lake network. Android and iOS support is a stretch goal. These are chosen due to the availability of inference engine support and hardware acceleration support for these operating systems.

P.3 Schedule and Milestones

Milestone	Due Date
Team formation and project selection	Sep 18, 2026
Problem statement, PoC plan, development plan, team charter	Sep 28, 2026
SRS and Hazard Analysis Rev 0	Oct 19, 2026
V&V Plan Rev 0	Nov 2, 2026
Design Document Rev -1 and CD-Fall	Nov 16, 2026
PoC demonstration	Nov 23–Dec 3, 2026
Design Document Rev 0	Jan 18, 2027
Peer module implementation	Jan 21, 2027
Rev 0 demonstration	Feb 1–11, 2027
V&V Report and CD-Winter	Mar 8, 2027
Final demonstration	Mar 15–21, 2027
Final documentation	Mar 31, 2027
EXPO poster and video	Mar 31, 2027 (tentative)
EXPO demonstration	April 6, 2027 (tentative)

Table 20: Project Milestones and Due Dates

P.4 Tasks and Deliverables

Task	Description / Steps	Deliverable	Due Date
Initial single-device prototype	Implement a model split up across two processes on a single device, communicating via inter-process communication.	V&V Plan Revision	Nov. 2
Expansion across two devices	Implement wired communication and model splitting (50% and 50%) across two devices, communicating via gRPC.	Design Document Revision	Nov. 16
Expansion across n devices & Partitioning algorithm	Implement model splitting for n devices through model partitioning that is aware of each device's performance and hardware limitations.	POC Demo	Nov. 23
Dynamic allocation	Implement algorithm to dynamically allocate model layers throughout a session.	Design Document Revision 0	Jan. 18
Test model allocation and dynamic add/drop	Test that system can handle devices being added or dropped from the network and that model split changes to maintain efficiency as network changes.	Design Document Revision 1	Jan. 18
User Interface Design & Network Display	Add user interface on top of API interface, listing statistics on model split and performance for each device.	Rev 0 Demo	Feb. 1
Integration & Communication Testing	Completion of integration testing to ensure sessions work and can handle various node failures.	Final Demo	Mar. 15
User & System Testing	Pass off the system to new users to ensure they intuitively understand how to control the system and are aware of its benefits.	Final Demo	Mar. 15
Final Delivery	Final tweaks to appearance, performance, and demonstration ahead of the Capstone EXPO.	Final Documentation	Mar. 31

Table 21: Tasks and Deliverables

Tasks that will be a part of each of the following entries include:

- **Unit Testing:** It is expected that each function and method will be tested in an isolated fashion. This is to ensure that each component works as intended and integration testing only reveals issues that originate from component to component interactions.
- **Documentation & Clean Code:** It is expected that each pull request or addition of new code will come with the relevant documentation and proper syntax. This is to alleviate cleanup work that is often left for the end of the project.

P.5 Required Technology Elements

- **Primary Programming Language** - The primary programming languages for the system will be Python and Rust. Python is chosen due to its common use for artificial intelligence applications and rich library ecosystem. Rust is chosen due to its high performance and type safety, along with it being a common choice for working directly with tensors. On each node, Python will be used for coordinator operations and Rust will be used for the inference engine. User interface will be created using TypeScript due to it being an industry standard type safe language for user interfaces.
- **Communication Framework** - gRPC is a remote procedure call framework developed by Google. It allows a client to call a method directly on a server and prioritizes speed and performance.
- **Inference Engine** - llama.cpp is an open-source LLM inference engine. It allows users to run large language models locally on consumer hardware. It was selected due to its ubiquity in the open-source AI field.
- **Machine Learning / Tensor Library** - ggml is a standard open-source library for machine learning inference on consumer hardware. Its focus on simplicity and broad backend support (across multiple kernels and platforms) is why the team selected it.
- **Model File Format** - GGUF is directly compatible with llama.cpp and ggml, so will be used as the format for transporting model layers from one device to another.

- **Frontend Framework** Next.js with TypeScript and Vite as the build tool. This is what the team is familiar with and aligns with the earlier choice for TypeScript as the frontend language.
- **Hardware** Multiple (>= 2) laptop or personal computer devices that can be connected via Thunderbolt to run local inference tests.
- **Version Control** Git, along with GitHub for source code management.
- **Continuous Integration** GitHub Actions for build pipelines and testing.
- **Testing API and Interface** OpenAI API Interface will be used for testing the system without using the user interface (or prior to building the user interface).

P.6 Risk and Mitigation Analysis

- **Risk: Lack of Performance Improvement when parallelising a model**
  - Description: This project focuses on the potential improvements of parallelising a model across a pool of devices. If in testing it is determined that splitting up a model is not yielding considerably better results in terms of speed or model benchmarks, the system may lose users.
  - Mitigation: Place additional focus on prioritising privacy of data and size of model that can be split up across a pool. This shifts the project's selling points from speed to privacy and quality and still achieves the original project goal.
- **Risk: Chosen Communication Technology (gRPC) fails to meet project needs**
  - Description: In the case gRPC does not meet project needs, the team's original strategy for inter-device communication will fail.
  - Mitigation: Perform initial research and record alternative options. Also research the core technology behind these libraries so that the team can make adaptations to fit a library to the project.
- **Risk: User Interface is Unclear**
  - Description: The user is unable to tell which devices are being used, or feedback about the device network is unclear to the user.

- Mitigation: Ensure that user testing and observation is completed by the team prior to the final delivery. This should be done by people unfamiliar with the product to ensure that controls and feedback are intuitive and easy to learn.

- **Risk: External Deliverables and Constraints Impacting Timeliness and Quality of Project Work**
  - Description: Due to various other commitments including work, other courses, and extra-curriculars, team members may not be able to complete high quality project work and deliverables on-time.
  - Mitigation: At the first meeting for a deliverable, the team will allocate tasks and assign issues to each member so workload is clear. As the team progresses through the deliverable, they may adjust the workload as needed, however should aim for an even split over the course of the project. The team will also establish an extended review meeting to conduct prior to deliverable hand-off to ensure it aligns with the rubric, the requirements, and their own quality standard.

P.7 Requirements Process and Report

Elicitation. The goals and metrics came from the Problem Statement and the Development Plan. The safety and security requirements came from the Hazard Analysis. Team 07 reviewed the Problem Statement and filed six issues. Two of them led to the privacy scope in NFR.G.4.1 and the measurable form of FR.G.4.4. Meetings with the teaching assistant led us to Meyer's template and its chapter structure. On October 16, 2026 we sent eight written questions to an AI developer who runs an in-house assistant at an aerospace company. His answers are in Appendix 1. The rest came from our own use of local inference tools, the distributed inference systems we studied, and the ggml, llama.cpp and OpenAI API documentation.

Communication. The team meets every Friday on Discord with a rotating chair and talks on the Discord server between meetings. We also meet the teaching assistant separately. Decisions that affect a document go into a GitHub issue or a pull request review. Issues are our only formal action items. A change agreed in a meeting or in chat is not final until it has an issue. Feedback from peer reviewers and the teaching assistant also comes in as issues. Each one is closed by a pull request or a written reason.

Change tracking. Each section of this document had its own issue (#94 to #131). A section was done when it reached level four of the course rubric, had its template note removed and built cleanly with `make`. Every change goes through a pull request that needs two approvals and must pass continuous integration. It builds every PDF and posts an automated rubric review. Requirement identifiers never change. New requirements take the next free number in their group and dropped ones are marked withdrawn. This keeps references from the Hazard Analysis, the V&V Plan and the design valid. Major revisions are recorded in Table 1. Each priority in Section S.7 shows the date it was last decided.

Process updates. Each recommended action in the Hazard Analysis FMEA is a safety and security requirement (SR0-1 to SR5-2). They are copied word for word into the Safety and Security Requirements section on page 80. Five of the 13 refine an FR.S or NFR.S requirement. The Hazard Analysis cites that requirement. The Hazard Analysis roadmap uses the milestone dates in Section P.3. A pull request that changes any of these edits both documents.

The V&V Plan is due November 2, 2026. It will use the 15 verification groups in Section S.8 as its test cases and the Symbolic Constants appendix for its thresholds. Every system, environmental, and safety and security requirement is in at least one group. A new requirement is not accepted without one. The Design Document is due November 16, 2026. It will trace each module to its requirements, and its modules must hide the likely changes in Section S.10. If a later document finds a gap, we fix it here first and then update that document.

Report. This revision has 12 high-level requirements in Section G.4, six functional and six non-functional. They are refined into 47 functional and 15 non-functional system requirements in Section S.2. With eight environmental and 13 safety and security requirements, there are 83 prioritised requirements. Of these, 58 are must, 21 should, four could and none deferred. There are eight use cases, six likely changes and four unlikely changes. About a dozen items trace to people outside the team. Two are the Team 07 issues above. The rest come from the user interview. It confirmed or changed FR.G.4.6, NFR.G.4.1, UC5, UC7, the device-owner stakeholder and the second persona. It also raised a no-telemetry requirement for S.2 that is still an open issue. The priorities were last decided on October 16, 2026. The peer review of this document is due October 22, 2026. We will handle its issues and the teaching assistant's feedback before writing the V&V Plan. Revision 1 is due with the final documentation on March 31, 2027.

Safety and Security Requirements

Functional Safety Requirements

- **SR0-1** – The system shall identify and discard stale device health data from determining device availability by configuring a time-to-live for valid data.
- **SR0-2** – The system shall only route inference requests to nodes that currently have a confirmed model layer partition.
- **SR1-1** – The system shall assign at least two nodes for model layer partitioning during distributed inference.
- **SR1-2** – The system shall ensure all model layers are assigned to a node, and that each device configured to run model layers receives a non-zero model layer partition assignment.
- **SR1-3** – The system shall reject models and layer allocations that are estimated to exceed the memory capacities of the Lake network or individual nodes, including a configured safety margin. Related SRS requirements: FR.S.2.1, FR.S.2.2.
- **SR2-1** – The system shall attempt an automatic bounded recovery and retry on inference requests if a follower node fails during inference. If the follower node is unable to be recovered within the configured recovery limits, the system must report to the API that the request has failed. Related SRS requirements: FR.S.1.10, FR.S.1.11, FR.S.1.13.
- **SR3-1** – The system shall identify corrupted and incomplete tensors throughout the distributed inference with integrity checks and communication timeouts to discard affected data and retry requests.
- **SR3-2** – The system shall log failed tensor decoding metadata and report a failure to the coordinator so the inference request can recover, subject to SR5-2.
- **SR4-1** – The system shall configure a timeout for communication between nodes, and handle exceeded timeouts according to the appropriate recovery or failure policy.
- **SR4-2** – The system shall mark a node as unavailable for inference or model layer partitioning when the node health criteria are not met, and require a successful health check before marking the same node as available. During a

leader node failure, follower nodes shall detect and initiate a shutdown. Related SRS requirements: FR.S.1.24 and FR.S.1.25.

- **SR4-3** – The system shall utilise configured values or historical data to determine a baseline inter-node communication latency and notify the coordinator when that baseline is exceeded by a specified tolerance.
- **SR5-1** – The system shall require user approval on the leader node to authorise a follower node, and user confirmation on the follower node to confirm the authorisation before it can participate in the Lake network. Related SRS requirements: FR.S.1.17, FR.S.1.18, FR.S.1.20, FR.S.1.21, NFR.S.12.
- **SR5-2** – The system shall not record prompts, responses, tensor data, and KV cache content in diagnostic logs. Related SRS requirements: NFR.S.10.

Requirement Dependencies

[A traceability diagram or matrix showing which requirements depend on which. Use the landscape environment below for a wide figure. —TPLT]

1 Appendix — User Interview

The questions below were sent, on October 16, 2026, to an AI developer at an aerospace engineering firm who runs the firm's in-house LLM assistant on its own hardware, and he answered in writing. The answers are summarised; the full notes are kept with the team's records. How they shaped the Goals chapter is described in Section G.7.2.

1. *What do you use to run models locally today, and on what machine?* vLLM for the main models, Ollama for small vision models, and a text-embeddings server, all in Docker on Linux, on two GPU workstations with 96 GB and 48 GB cards. The main model is a 27-billion parameter model at full precision with a 131k-token context.

2. *What stops you from running a bigger model?* Memory per card: the 27B model fills most of a 96 GB card once conversation context is added, and a 70B-class model would need both cards or compression. Also prompt-reading time on long agent prompts, and leaving room for concurrent users. Model quality is already good enough for their tasks.

3. *Do you have idle computers, and would you pool them?* Most office machines are laptops with integrated graphics; some GPU workstations sit idle at night. Yes for background jobs (re-indexing, OCR, embeddings, overnight test runs); probably not for live chat, since a split over the office network would likely be slower than one good GPU. Conditions: data control on every machine, backing off instantly when the owner is working, tolerance for machines sleeping or updating, network speed, and IT approval with signed installers and an off switch.

4. *If responses were slow, what would you want to see?* A per-request timeline per device with the slowest highlighted, per-link bandwidth and latency, a comparison against the single best machine, whether the prompt cache was hit, and metrics on by default and exportable.

5. *If a machine dropped out mid-answer?* Retry quietly once, continuing from where it stopped; if that fails, tell the user and show what was written so far. Never return a cut-off answer that looks complete, and never repeat an action, since their agent drafts emails and edits files. Log every drop-out and remove machines that keep failing.

6. *How much does keeping data on your network matter?* It is the reason they run locally: cloud models cannot legally process export-controlled documents. Anything like Lake must run fully offline with no telemetry, encrypt traffic between nodes, accept only company-managed machines, and be auditable. Shared or public networks: no.

7. *Which tools would you use it from?* An OpenAI-compatible API is a must, with streaming, tool calling, strict JSON output, image input, and options such as turning "thinking" off. Then a chat window, a coding assistant for code that cannot go to the cloud, and batch scripts.

8. *Command line or installer and UI?* Command line and Docker on servers, with configuration in files and pinned versions. On colleagues' machines, a signed installer pushed by IT, running as a background service with a pause switch. A small admin page for machine health and timing. No automatic updates.

2 Appendix — Symbolic Constants

Tables 22 and 23 give the value of each named constant used in this document. The values are initial estimates and are expected to change as testing gives better data.

Table 22: Constants used in the system requirements

Name	Value	Description
HEALTH_INTERVAL	5 s	Time between health and performance reports. Used in FR.S.1.2, FR.S.1.3, FR.S.1.23, FR.S.3.4, FR.S.4.5.
LEADER_TIMEOUT	30 s	Time without a health query after which a follower assumes the leader has failed. Used in FR.S.1.24.
FAIL_THRESHOLD	50%	Fraction of a node's usual token throughput below which it is treated as failing. Used in FR.S.1.10.
UI_TASK_TIME	2 min	Time a new user may take to complete a usability task. Used in NFR.S.1, NFR.S.2.
INSTALL_CLICKS	10	Maximum clicks to install Lake on Windows or macOS. Used in NFR.S.3.
MAX_DEGRADATION	20%	Maximum loss of per-request throughput when serving concurrent requests. Used in NFR.S.5.
MIN_CONCURRENT	2	Minimum number of requests every cluster must serve at the same time. Used in S.8 group 13 and the Validation list.
BENCH_RATIO	95%	Minimum benchmark score of a split model, relative to the unsplit model. Used in NFR.S.6.
MAX_LOSS	10%	Packet loss the system must tolerate while still producing correct output. Used in NFR.S.9.
CONTEXT_MAX	4096 tokens	Longest context (prompt and response) a session can use. Each node reserves KV cache for this many tokens. Used in S.5, S.9.

Table 23: Constants used in the safety requirements and verification

Name	**Value**	**Description**
MEMORY_MARGIN	10%	Share of each node's memory left free for runtime overhead. Used in FR.S.2.1, SR1-3.
HEALTH_TTL	15 s	Age after which health data is stale and no longer used to judge availability. Used in SR0-1.
RECOVERY_LIMIT	3	Maximum recovery attempts for one request before it fails. Used in SR2-1.
NODE_TIMEOUT	10 s	Time without a response after which a connection between nodes is treated as failed. Used in SR4-1.
LATENCY_TOL	50%	Increase over a connection's baseline latency that is reported to the Coordinator. Used in SR4-3.
INTERVAL_TOL	1 s	Allowed deviation from *HEALTH_INTERVAL* when verifying reports. Used in S.8 group 5.
TEST_USERS	3	Minimum number of participants in the usability session. Used in S.8 group 10.

3 Appendix — Generative AI Use Disclosure

3.1 AI Tools Used

- Anthropic Claude Sonnet 5.5, Opus 5.5 and Fable 5.1, used through Claude Code 2.1.303.
- OpenAI GPT-6.1 Sol, used to ideate and plan sections.
- Google Gemini 3.5 Flash Lite, used by the automated rubric review action in the repository.

3.2 How and Where Used

We used AI to help ideate and plan every section. The requirements, their rationale and their priorities were written by the team. AI was also used in the following places.

- **Goals (G.1–G.8)**: Claude drafted the chapter from the Problem Statement, the Development Plan, our Discord decisions and the user interview notes. It also wrote the interview questions and rendered the context diagram, goal model and use case diagram from sources we specified.
- **S.6 Detailed Usage Scenarios**: Claude expanded the G.5 use cases into full scenarios. It proposed the actors and steps and we edited the result.
- **S.9 Formalisations**: AI helped write the formulas.
- **NFR.S.11, P.7, Requirement Dependencies, References, the Glossary and the appendices**: on 2026-10-19 the whole team sat together and told Claude Code what to write. It wrote the NFR.S.11 wording, P.7, the Requirement Dependencies section and its table script, the References clean-up, the reflection draft and this disclosure. It also linked glossary terms and requirement IDs and made terminology consistent.
- **All chapters**: Gemini posts rubric comments on every pull request that changes the documents. We decide what to change.

3.3 Verification of AI-Generated Content

Every pull request needs at least two team approvals before it merges. We checked every requirement ID in AI-drafted text against the requirement it names. We checked the interview summary in the Goals chapter against the written answers in the interview appendix. The dependency table is generated from the document source. The team edited the reflection draft so every statement is confirmed by a team member. Each author checked this appendix for their own sections. The team is responsible for the accuracy, completeness, consistency and appropriateness of this document, whether a person or an AI tool wrote it.

4 Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. How many of your requirements were inspired by speaking to your client(s) or their proxies (e.g. your peers, stakeholders, potential users)?

4. Which of the courses you have taken, or are currently taking, will help your team to be successful with your capstone project.

5. What knowledge and skills will the team collectively need to acquire to successfully complete this capstone project? Examples of possible knowledge to acquire include domain specific knowledge from the domain of your application, or software engineering knowledge, mechatronics knowledge or computer science knowledge. Skills may be related to technology, or writing, or presentation, or team management, etc. You should look to identify at least one item for each team member.

6. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

1. Reviewing went well. Every section had its own pull request with two approvals, and we read the rendered PDF instead of the diff. Writing the Hazard Analysis at the same time also helped, as creating the FMEA helped with writing our safety requirements.

2. Our biggest pain point was switching SRS templates. After a meeting with our TA on October 9th, we switched from the Volere template to the Meyer template, requiring us to re-do significant administrative work. Five people contributing to the same file also brought with it some merging headaches. In particular, moving the glossary and adding to the template changes list caused some annoying merge conflicts.

3. We got outside input from three places. Team 07 peer reviewed our Problem Statement, we met with our TA, and we did one written user interview on October 16th. Of our detailed requirements, only five came directly from the peer review or the interview. Team 07 asked what dynamic allocation means in measurable terms, which became FR.S.1.10 to FR.S.1.13, and the interview made no telemetry a hard requirement in NFR.S.11. Most of the rest came from our own use of llama.cpp and Ollama as well as from the Hazard Analysis. One interview is likely not enough, so we plan to talk to more local AI users.

4. We've all taken the core Software Engineering courses. SFWRENG 3RA3 taught us goal models and the Meyer template, and 3SH3 covers the memory and IPC ideas behind the node daemon. This term the team is also taking 4AL3. Being our first machine learning course, it will be integral to helping us build a model partitioning algorithm.

5. The core learning that will need to be made among the team is surrounding inference engines. Since the team has minimal experience with inference engines, this will require up-front knowledge and research to become familiar with the tools. Jinwoo and Colin, the machine learning leads, will need to learn the fundamentals of ggml and the GGUF format. Gary, the IPC lead, will need to build up a solid foundation of knowledge on gRPC as well as conceptual knowledge for how the system will handle device health checks and failure accommodation. Matt, the performance lead, will need more learning related to benchmarks and GPU profiling to ensure we can competently benchmark our system and identify areas for improvement. Finally, Ethan, the architecture lead, will need to learn from other distributed inference systems to find the benefits and pitfalls of previous approaches and what will apply to Lake.

6. Jinwoo and Colin: Read the ggml source first or build a quick proof of concept that splits one model across two processes. These members will pursue the proof of concept first to become more familiar with ggml, as well as the fact that the project POC will require ggml related code. Gary's options include working through the gRPC tutorials, or directly moving to prototyping inter-device health checks, referencing documentation as needed. The prototype is Gary's choice in this case, since it will once again be helpful for the project as a whole and will give him the relevant knowledge more quickly. Matt has the option to study profiler documentation or build a small benchmark harness locally to test his skills and identify knowledge gaps. He has decided to move forward with building the benchmark harness to compare against a single-device llama.cpp baseline. Lastly, Ethan is faced with the choice to either read papers on distributed inference and general strategy, or look to existing providers such as Exo to see how they coordinate their systems. He will also opt for option 2, since this allows the team to avoid pitfalls that Exo fell in to, while also identifying key areas that Lake may differentiate itself from existing providers.